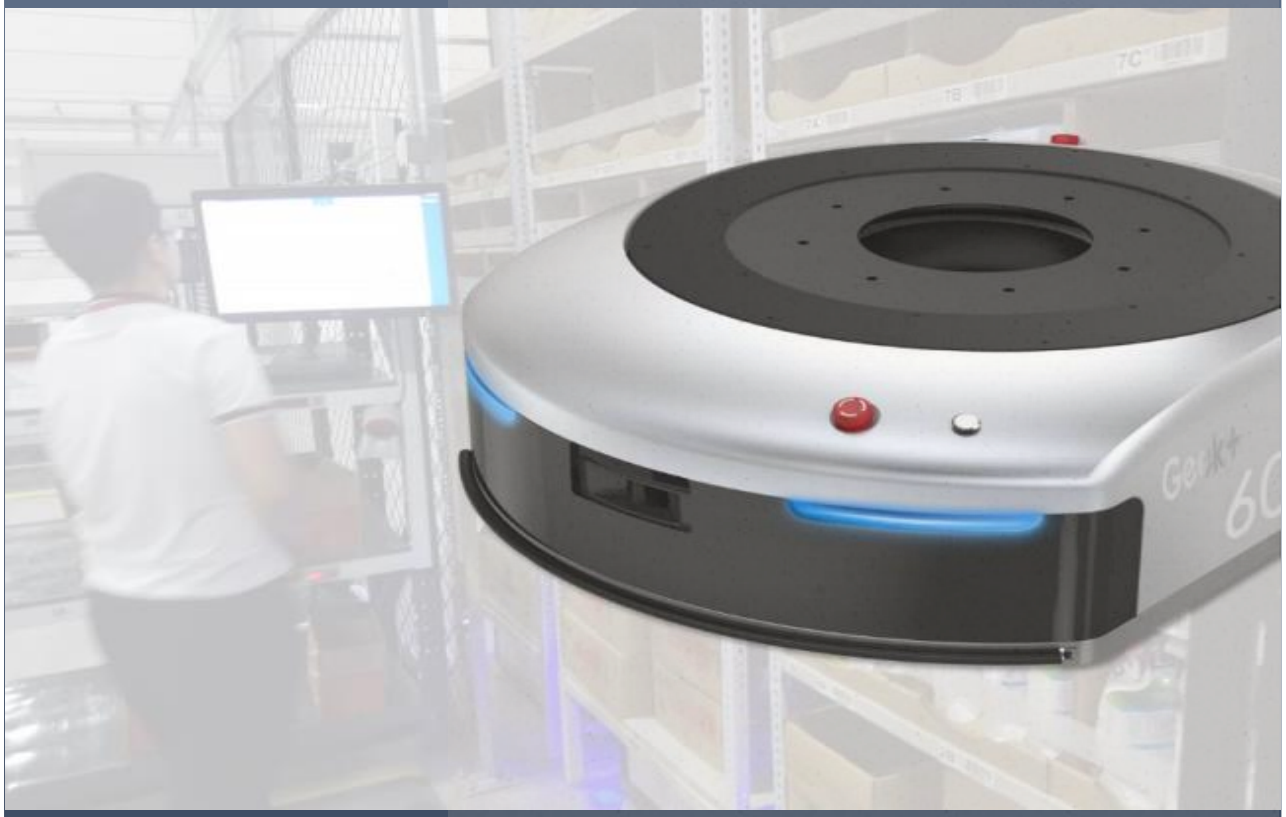
# Geek+ GMS Interface Design Document

Interface Design Document



## GEEK+ INTERFACE DESIGN DOCUMENT

APPLICABLE TO: GMS 3.2.0

GEEKPLUS TECHNOLOGY CO., LTD.

# Contents


1 Overview ....4
1.1 Basic Concepts ....4
1.2 Custom Data Type ....4
2 Overall Description ....5
2.1 Message ....5
2.2 Message Type ....5
2.3 Message Structure ....6
2.3.1 Request Message ....7
2.3.2 Response Message ....9
2.4 Request Principle ....11
2.5 Response Description ....16
2.6 Task Callback and Response ....17
3 Protocol Description ....17
3.1 Interface Access Methods ....17
3.1.1 TCP SOCKET Protocol ....17
3.1.2 HTTP Protocol ....19
3.2 Message Specification ....20
4 Business Description ....20
5 Interface Logs ....26
6 Interface Description ....27
6.1 Task Interfaces ....27
6.1.1 Point-to-point Moving Interface - MovingRequestMsg ....27
6.1.2 Interface of Moving between Dynamic Points - DynamicMovingRequestMsg ....33
6.1.3 Interface of Moving from a Point to Multiple Points - MultiMovingRequestMsg ....39
6.1.4 Dynamic Unit Template Moving Interface - DynamicUnitMovingRequestMsg ....45
6.2 Task Flow (Workflow) Interfaces ....51
6.2.1 Task Flow (Workflow) Start Interface - WorkflowStartMsg ....51


This document shall not be disseminated in any form without the written permission of Geek+.

6.2.2 Task (Workflow) Next-step Interface - InstanceOperationMsg - NEXT_STEP ..... 58
6.2.3 Workflow Query Interface - WorkflowListMsg ..... 62
6.2.4 Workflow Instance (task) Query Interface - WorkflowInstanceListMsg ..... 67
6.2.5 Forced Task Flow Deletion Interface (Workflow Cancellation) - InstanceOperationMsg - WORKFLOW_DELETE ..... 74
6.2.6 Interface of Task Flow Cancellation and Destination Change (Workflow Cancellation and Delivery of Goods to the Designated Point) - InstanceOperationMsg - WORKFLOW_DROP ..... 77
6.2.7 Interface for Proceeding to Next Step of Task at Wait Point - InstanceOperationMsg - CLEAR_WAIT ..... 81
6.2.8 Task Flow (Workflow) Priority Adjustment Interface - InstanceOperationMsg - PRIORITY_ADJUSTMENT ..... 84
6.2.9 Container Turning Interface - InstanceOperationMsg - GO_TURN ..... 87
6.2.10 Robot Stop Interface - InstanceOperationMsg - STOP ..... 91
6.2.11 Robot Recovery Interface - InstanceOperationMsg - ROBOT_RECOVER ..... 95
6.2.12 Cell Locking/Unlocking Interface - InstanceOperationMsg - CELL_CODE ..... 98
6.3 Container (Shelf) Interfaces ..... 100
6.3.1 Container (Shelf) Query Interface - ContainerListMsg ..... 100
6.3.2 New Container Interface - ContainerOperationMsg ..... 103
6.3.3 Container (Shelf) Addition Interface - ContainerOperationMsg ..... 108
6.3.4 Container (Shelf) Removal Interface - ContainerOperationMsg ..... 111
6.4 Workstation Interfaces ..... 113
6.4.1 Interface for Starting Queuing for Receiving Goods - StationQueueMsg - START ..... 113
6.4.2 Interface for Stopping Queuing for Receiving Goods - StationQueueMsg - STOP ..... 115
6.4.3 Workstation Query Interface - StationListMsg ..... 117
6.5 Map and Robot Resource Interfaces ..... 120

This document shall not be disseminated in any form without the written permission of Geek+.

6.5.1 Area Query Interface - AreaListMsg ..... 120
6.5.2 Location Query Interface - LocationListMsg ..... 126
6.5.3 Robot Information Query Interface - RobotInfoMsg ..... 137
6.5.4 Charging Station Information Query Interface - ChargerInfoMsg . 145
6.5.5 Map Information Query Interface - MapInfoMsg ..... 148
6.5.6 Fire Emergency Stop Interface - RobotControlMsg ..... 156
6.6 Traffic Control Area Interfaces ..... 159
6.6.1 Traffic Control Area Query Interface - TrafficControlAreaListMsg ..... 159
6.6.2 Interface of Applying for Entering/Leaving Traffic Control Area (Synchronous Interface) - CrossAreaPermissionMsg ..... 163
6.6.3 Interface of Applying for Entering/Leaving Traffic Control Area (Asynchronous Interface) - CrossAreaPermissionBMsg ..... 166
6.6.4 Interface of Canceling Application for Entering/Leaving Traffic Control Area - CrossAreaCancelMsg ..... 169
6.6.5 Interface of Approval of Entering/Leaving Traffic Control Area through Asynchronous Interface - CrossAreaApproveMsg ..... 171
6.6.6 Passing Status Feedback Interface - AreaMovingPhaseMsg ..... 174
6.6.7 Traffic Control Area Locking/Releasing Interface - TrafficAreaLockMsg ..... 176
6.7 Callback Interface ..... 178
6.7.1 Task Callback Interface - MovingCallbackMsg ..... 178
7 Usage Rules ..... 187
7.1 Area ..... 187
8 Reason Codes ..... 188
8.1 Reason Codes ..... 188
8.2 GMS Management Exception Codes ..... 198
8.3 Robot Exception Codes ..... 200
8.4 System Exception Codes ..... 206

This document shall not be disseminated in any form without the written permission of Geek+.

# 1 Overview

This document is prepared to define the general external service interfaces of the enterprise service platform of Geek+ Moving System (GMS), to integrate with the customer's system rapidly, enable the transmission of information flow between the customer's system and GMS, promote the customer's system to become more intelligent and automated, and thus achieve a win-win situation of both enterprises.

From the platform perspective, the interface design follows the principle of minimum development workload for customers and integrates logic processing into GMS, to allow customers to focus on their own businesses, drive GMS by their own means, and perform a series of robot operations.

## 1.1 Basic Concepts

| Concept | Introduction | Remarks |
|---|---|---|
| **Host system** | The user custom application (namely upstream program system of GMS) that sends task messages to GMS is collectively called the host system in this document. | |

## 1.2 Custom Data Type

Except for the eight basic types of JAVA and the String type, some data types are customized by GMS. The user of the host system needs to pay attention when entering or parsing such types of data.

The following table lists typical types of GMS custom data. Users can refer to the related description in this document or consult Geek+ technical staff when entering or parsing such data.

This document shall not be disseminated in any form without the written permission of Geek+.

| Custom data | Meaning | Type description | Example (value) |
|---|---|---|---|
| **DPoint** | It refers to the decimal three-dimensional coordinates, where "z" represents the floor, and "x" and "y" represent the plane coordinates. | "z" is an integer, while "x" and "y" are double. | {"z": 1,"x": 29.5,"y": 10.5} |
| **IPoint** | It refers to the integral three-dimensional coordinates, where "z" represents the floor, and "x" and "y" represent the plane coordinates. | Both "x", "y", and "z" are integers. | {"z":1,"x":45,"y":42} |

# 2 Overall Description

## 2.1 Message

Each communication between the host system and GMS is referred to as a message. A message is a set of data structure, including message type, message header, and message body, defined by GMS for communication with external systems.

## 2.2 Message Type

According to the message direction, messages are divided into six types, as shown in the table below:

| Message type | Function description | Message direction |
|---|---|---|
| **Request message** | A request message is sent by an an external system to GMS. Request messages include distributing tasks and instructions, task and instruction updates, | Host system -> GMS |

This document shall not be disseminated in any form without the written permission of Geek+.

| Message type | Function description | Message direction |
|---|---|---|
| | and query instructions. | |
| **Response message** | A response message is sent by GMS to respond to a request from the host system. | GMS -> host system |
| **Callback request message** | GMS will send an asynchronous callback request message to the external system in the case of change in the task phase and robot operation. | GMS -> host system |
| **Callback response message** | The delivery of callback messages of GMS is strictly guaranteed. Therefore, the host system must respond with a callback response message upon receiving a callback message. | Host system -> GMS |
| **Heartbeat message** | The heartbeat message for keeping persistent connection only applies to the scenario where the host system creates a connection with GMS based on TCP SOCKET (regardless of other protocols). | Host system -> GMS |
| **Response message to unknown information** | When receiving a message which cannot be correctly parsed, GMS will send a response message to unknown information to the host system (in this case, the technical staff of the host system needs to check the validity of the message). | GMS -> host system |

## 2.3 Message Structure

GMS API uses the standard JSON message format. Different messages are also slightly different in structure.

This document shall not be disseminated in any form without the written permission of Geek+.

### 2.3.1 Request Message

The host system shall send a request message to GMS to implement some functions.

For example, when the host system wants GMS to implement the point-to-point moving task, the host system shall send a **point-to-point moving task message** to GMS. Upon receiving the message, GMS will perform validity verification. After the message passes verification, RMS will send a **response message** to inform the host system that it has received the message; otherwise, GMS will also send a **response message** to inform the host system that the task request message is wrong.

Or when the host system wants to know the information of certain resources, it can send a **query request message** to GMS. Upon receiving the message, GMS will send the resource information in the form of **response message** to the host system.

The format of the request message is shown below:

```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "language": "zh_cn"
    },
    "body": {
        "msgType": "MovingRequestMsg",
        "field1": 1,
        "field2": "xxx"
```

This document shall not be disseminated in any form without the written permission of Geek+.

```
        }
    }
```

Field description:

| Field | Required | Function description |
|---|---|---|
| **header** | Yes | It represents the message header. It is defined in the request message, containing the basic verification information field. |
| **requestId** | Yes | RequestId is included in the message header to uniquely identify a request and its response.<br>The value of this field is defined by the host system, and the response message shall have the same value.<br>GMS shall guarantee idempotency based on this ID. Therefore, the ID shall be globally unique. UUID is recommended. |
| **clientCode** | Yes | ClientCode is the identity of the upstream customer and is contained in the message header, representing the user code.<br>The clientCode is issued by GMS to the upstream customer and consistent in the same project. |
| **channelId** | Yes | ChannelId is the identity of the link channel and contained in the message header. |
| **requestTime** | Yes | It represents the request sending time in the format of yyyy-MM-dd hh:mm:ss. |
| **language** | No | Language is contained in the message header and used to set the language of the response message.<br>Based on this set field, RMS will select the language in |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Required | Function description |
|---|---|---|
| | | which the response message is sent.<br>Currently, the system supports simplified Chinese (zh_cn), English (en_us), Japanese (ja_jp), and traditional Chinese (zh_hk/zh_tw).<br>**The default language is simplified Chinese (zh_cn) if this field is not passed.** |
| **body** | | It represents the message text and its detailed fields.<br>The subsequent sections will elaborate on the request messages of different types. |

## 2.3.2 Response Message

For each valid request message from the host system, GMS will send a response message as a response.

The format of the response message is shown below:

```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "MovingRequestMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "field1": "xxx",
        "field2": "xxx"
```

```
    }
}
```

Field description:

| Field | Function description |
|---|---|
| **header** | It represents the message header. It contains the basic verification information field. |
| **responseId** | ResponseId is contained in the message header and is consistent with the requestId of the request message corresponding to the current response message. |
| **clientCode** | ClientCode is the identity of the upstream customer and is contained in the message header, representing the user code. The clientCode is issued by GMS to the upstream customer and consistent in the same project. |
| **requestTime** | It represents the request sending time in the format of yyyy-MM-dd hh:mm:ss. |
| **msgType** | It represents the type of the current response message, corresponding to the type of the callback message. |
| **code** | Code is the response code and contained in the message header.<br>For detailed response codes and message descriptions, see the appendix. |
| **msg** | Msg describes the response message and is contained in the message header.<br>This value is used to describe the response to the request message. |
| **body** | It represents a message text with detailed fields.<br>Some response messages do not have the text field. |

This document shall not be disseminated in any form without the written permission of Geek+.

## 2.4 Request Principle

GMS determines the unique message request based on the requestId in the message. When GMS receives multiple requests of the same requestId, it will respond with identical response message, even though the contents of the two request messages are completely different.

**Therefore, when sending messages to GMS, the host system must use different requestIds. UUID is recommended.**

For example, the host system resends an identical point-to-point moving task request 15 min after sending the same task.

First request:


```
{
    "header": {
        "requestId": "b4134151-81c9-4ab0-808c-be20ffc1d074",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2022-10-21 16:20:24"
    },
    "body": {
        "msgType": "MovingRequestMsg",
        "taskType": "M000163",
        "taskCode": "",
        "locationFrom": "4020",
        "containerCode": "",
        "containerCategory": "",
        "locationTo": "4016",
        "priority": 1
    }
```

```
}
```

Response:


```
{
    "header": {
        "responseId": "b4134151-81c9-4ab0-808c-be20ffc1d074",
        "msgType": "MovingRequestMsg",
        "code": "0",
        "requestTime": "2022-10-21 16:20:24",
        "clientCode": "geekplus",
        "msg": "调用成功"
    },
    "body": {
        "workflowCode": "F202211020000007",
        "taskCode": "",
        "instanceId": 238,
        "instancePriority": 1,
        "containerCode": "00000112"
    }
}
```


**Error** example of the second request: (identical to the first request)


```
{
    "header": {
        "requestId": "b4134151-81c9-4ab0-808c-be20ffc1d074",
        "channelId": "client-01",
        "clientCode": "geekplus",
```

```
        "requestTime": "2022-10-21 16:20:24"
    },
    "body": {
        "msgType": "MovingRequestMsg",
        "taskType": "M000163",
        "taskCode": "",
        "locationFrom": "4020",
        "containerCode": "",
        "containerCategory": "",
        "locationTo": "4016",
        "priority": 1
    }
}
```


Response:


```
{
    "header": {
        "responseId": "b4134151-81c9-4ab0-808c-be20ffc1d074",
        "msgType": "MovingRequestMsg",
        "code": "0",
        "requestTime": "2022-10-21 16:20:24",
        "clientCode": "geekplus",
        "msg": "调用成功"
    },
    "body": {
        "workflowCode": "F202211020000007",
        "taskCode": "",
```

```
            "instanceId": 238,
            "instancePriority": 1,
            "containerCode": "00000112"
        }
}
```


或


```
{
    "header": {
        "responseId": "b4134151-81c9-4ab0-808c-be20ffc1d074",
        "msgType": "MovingRequestMsg",
        "code": "1006",
        "requestTime": "2022-10-21 16:20:24",
        "clientCode": "geekplus",
        "msg": "RequestId  already exists!"
    },
    "body": {}
}
```


There are two situations at this point:

1. To guarantee idempotency, the first response message will be directly returned, and no task will be truly distributed.

2. If existing response message has not been found in GMS, 1006 error code will be reported.

**Correct** example of the second request:


```
{
    "header": {
        "requestId": "f1d82977-1824-4f33-b5c1-cd67f6d407ef",
```

```
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2022-10-22 16:20:24"
    },
    "body": {
        "msgType": "MovingRequestMsg",
        "taskType": "M000163",
        "taskCode": "",
        "locationFrom": "4020",
        "containerCode": "",
        "containerCategory": "",
        "locationTo": "4016",
        "priority": 1
    }
}
```


Response:


```
{
    "header": {
        "responseId": "f1d82977-1824-4f33-b5c1-cd67f6d407ef",
        "msgType": "MovingRequestMsg",
        "code": "0",
        "requestTime": "2022-10-21 16:20:24",
        "clientCode": "geekplus",
        "msg": "调用成功"
    },
    "body": {
        "workflowCode": "F202211020000007",
```

```
            "taskCode": "",
            "instanceId": 239,
            "instancePriority": 1,
            "containerCode": "00000112"
        }
    }
```


## 2.5 Response Description

Typically, GMS will send a response message to the host system immediately after message verification, regardless of whether the verification is passed or not (the response message is probably sent in milliseconds, excluding the impact of network latency). If no response message is received from GMS after a long time (such as 30 s), the host system shall resend the request message.

After the request message passes verification and is implemented, GMS will send a response message with a code of "0" and a description of "Success" to the host system, which indicates that it has received and implemented the request.

When the request message fails to pass verification by GMP, GMS will send the response message with possibly different codes other than 0. Each response code and message description will indicate the reason why the current request message fails to pass verification. The customer shall modify the request message of the host system accordingly, and send the request message again.

Therefore, after receiving the response message from GMS, the host system needs to determine the request state according to the "code" field in the response message. The code "0" indicates that the request succeeded, and

This document shall not be disseminated in any form without the written permission of Geek+.

a code other than "0" indicates that the request failed.

## 2.6 Task Callback and Response

When the implementation state or implementation phase of a task changes, GMS will send a callback message to the host system. Upon receiving the message, the host system shall send a response message to inform GMS that it has received the message. At this point, a life cycle of a callback message is completed.

If failing to receive the response message within 15 s, GMS might think message sending failed and will send the callback message again. If still failing to receive the response message after 5 min, GMS will stop retry.

The callback message is actually a request made by GMS to the host system. As mentioned above, if the host system creates a connection with GMS via HTTP protocol, an HTTP server shall be established to receive messages for synchronous communication between the two.

If the host system uses TCP SOCKET and WEBSOCKET protocols, GMS will, as a server, directly send the callback message to the client when the task state changes.

# 3 Protocol Description

## 3.1 Interface Access Methods

The host system can establish a connection with GMS via TCP SOCKET protocol or HTTP POST protocol, but not simultaneously.

### 3.1.1 TCP SOCKET Protocol

When using TCP SOCKET, the host system acts as the client and GMS acts

This document shall not be disseminated in any form without the written permission of Geek+.

as the server. All request messages, response messages, callback messages, and heartbeat messages are sent through the same SOCKET channel.

GMS splits data packets with the carriage return character and new line character ("\r\n"). This means that all messages sent by the host system to GMS shall end with the separator character "\r\n" without the carriage return character and new line character in the message body. All messages sent by GMS will end with the separator character "\r\n".

a) Port Number

GMS uses port 24245 for communication via SOCKET by default.

b) Persistent Connection

For TCP SOCKET, GMS as the server will actively detect the connection status of TCP. If the client does not send any message to GMS after more than 30 s, GMS will disconnect the TCP connection with the client. The client also needs to reconnect with TCP when it has not received the message sent by GMS after a long time.

Therefore, when the client is in a prolonged writing-idle period, it needs to send heartbeat messages to GMS through the current TCP channel. This writing-idle period is recommended to be set as 20 s. In other words, when the client is in the writing-idle period for more than 20 s, it shall send heartbeat messages to GMS.

Example of a heartbeat message sent by the host system to GMS:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekcode",
        "channelId": "11111",
        "userId": "admin",
        "userKey": "123456",
```

```
            "language": "zh_cn",
            "version": "1.0.0"
        },
        "body": {
            "msgType": "HeartbeatMsg"
        }
    }
```

### 3.1.2 HTTP Protocol

HTTP doesn't support full-duplex communication. Therefore, both GMS and the host system shall establish a HTTP server for each other to receive and process messages under the HTTP protocol. When the host system sends a request message, the host system acts as the client and GMS acts as the server. When GMS sends a callback message, the host system acts as the server and GMS acts as the client.

When using HTTP, the host system must access GMS through POST in the HTTP request, and the user shall set the MIME type in the HTTP header as "application/json". Similarly, GMS shall send response messages of the "application/json" type to the host system. Response messages shall be sent to the host system in the same HTTP session.

a) Port Number

GMS uses port 24249 for communication via HTTP by default.

b) Persistent Connection

After the communication of the request message and response message is completed, the HTTP session will not be retained. Therefore, the host system needs to create an HTTP server to listen to request messages, receive callback messages from GMS, and send callback response messages to

This document shall not be disseminated in any form without the written permission of Geek+.

GMS.

GMS will resend a request message about every 15 s within 5 min after the request message failed to be sent, and stop sending the request message after 5 min.

## 3.2 Message Specification

- Messages are organized in JSON format.
- The time is represented by UTC time in Long type, accurate to milliseconds.
- Input parameters are strictly case-sensitive.

# 4 Business Description

| Business scenario | Service conditions | Service interface |
|---|---|---|
| **Container (shelf) moving between two points** | The host system manages shelves or locations (stations). | Point-to-point moving interface-MovingRequestMsg<br>Task cancellation interface-InstanceOperationMsg<br>Task priority adjustment interface - InstanceOperationMsg<br>Forced task cancellation interface (workflow cancellation and delivery of goods to the designated point) - InstanceOperationMsg<br>Task callback interface - MovingCallbackMsg |
| **Container (shelf) moving** | The host system manages shelves or | Interface of moving between dynamic points - DynamicMovingRequestMsg |

| Business scenario | Service conditions | Service interface |
|---|---|---|
| **between dynamic points** | locations (stations). | Task cancellation interface-InstanceOperationMsg<br>Task priority adjustment interface - InstanceOperationMsg<br>Forced task cancellation interface (workflow cancellation and delivery of goods to the designated point) - InstanceOperationMsg<br>Task callback interface - MovingCallbackMsg |
| **No-load robot operation between two points** | The host system manages locations or stations. | Point-to-point moving interface-MovingRequestMsg<br>Task cancellation interface-InstanceOperationMsg<br>Task priority adjustment interface - InstanceOperationMsg<br>Task callback interface - MovingCallbackMsg |
| **Shelf moving between multiple points** | The host system manages shelves or locations. | Interface of moving from a point to multiple points - MultiMovingRequestMsg<br>Task next-step interface - InstanceOperationMsg<br>Task cancellation interface-InstanceOperationMsg<br>Task priority adjustment interface - InstanceOperationMsg<br>Forced task cancellation interface (workflow |

This document shall not be disseminated in any form without the written permission of Geek+.

| Business scenario | Service conditions | Service interface |
|---|---|---|
| | | cancellation and delivery of goods to the designated point) - InstanceOperationMsg<br>Task callback interface - MovingCallbackMsg |
| | The host system only manages tasks, but not shelves or locations. | Workflow list query interface - WorkflowListMsg<br>Workflow start interface - WorkflowStartMsg<br>Workflow next-step interface - InstanceOperationMsg<br>Workflow cancellation interface - InstanceOperationMsg<br>Workflow priority adjustment interface - InstanceOperationMsg<br>Task callback interface - MovingCallbackMsg |
| **No-load robot operation between multiple points** | The host system manages locations or stations. | Interface of moving from a point to multiple points - MultiMovingRequestMsg<br>Task next-step interface - InstanceOperationMsg<br>Task cancellation interface- InstanceOperationMsg<br>Task priority adjustment interface - InstanceOperationMsg<br>Forced task cancellation interface (workflow cancellation and delivery of goods to the designated point) - InstanceOperationMsg |

This document shall not be disseminated in any form without the written permission of Geek+.

| Business scenario | Service conditions | Service interface |
|---|---|---|
| | | Task callback interface - MovingCallbackMsg |
| | The host system only manages tasks, but not locations. | Workflow list query interface - WorkflowListMsg<br>Workflow start interface - WorkflowStartMsg<br>Workflow next-step interface - InstanceOperationMsg<br>Workflow cancellation interface - InstanceOperationMsg<br>Workflow priority adjustment interface - InstanceOperationMsg<br>Task callback interface - MovingCallbackMsg |
| **Shelf pickup at workstation (station)** | The host system manages neither shelves nor shelf inbound locations. Robots pick up shelves in special warehouse areas. | Workflow start interface - WorkflowStartMsg<br>Task cancellation interface- InstanceOperationMsg<br>Task priority adjustment interface - InstanceOperationMsg<br>Task callback interface - MovingCallbackMsg |
| **Shelf delivery to workstation (station)** | The host system manages neither shelves nor shelf storage locations. Robots deliver shelves to special | Workflow start interface - WorkflowStartMsg<br>Task cancellation interface- InstanceOperationMsg<br>Task priority adjustment interface - InstanceOperationMsg<br>Task callback interface - |

This document shall not be disseminated in any form without the written permission of Geek+.

| Business scenario | Service conditions | Service interface |
|---|---|---|
| | warehouse areas. | MovingCallbackMsg |
| **Container addition to the system** | The host system manages shelves. | New container interface - ContainerOperationMsg |
| **Manually moving shelves to designated positions** | The host system manages locations or shelves. | Container (shelf) entry interface - ContainerOperationMsg |
| **Manually moving shelves from a certain location** | The host system manages locations or shelves. | Container (shelf) exit interface - ContainerOperationMsg |
| **Robot standby in advance** | Under the condition that the cycle time of delivery is met, robots arrive at nearby areas for standby in advance. | Workstation list query interface- wwStationListMsg<br>Interface of starting station queue - StationQueueMsg<br>Interface of stopping station queue - StationQueueMsg |
| **Query on real-time conditions of traffic control area** | The host system does not perform traffic control. | Interface for query on real-time conditions of traffic control area - TrafficControlAreaListMsg |
| **Application for entering/leaving traffic control area** | The host system does not perform traffic control. | Interface of application for entering traffic control area - CrossAreaPermissionMsg<br>Interface of application for leaving traffic control area - CrossAreaPermissionMsg |

This document shall not be disseminated in any form without the written permission of Geek+.

| Business scenario | Service conditions | Service interface |
|---|---|---|
| | | Interface of application for entering traffic control area asynchronously - CrossAreaPermissionBMsg<br>Interface of application for leaving traffic control area asynchronously - CrossAreaPermissionBMsg |
| **Cancellation of application for entering/leaving traffic control area** | The host system does not perform traffic control. | Interface of cancellation of application for entering traffic control area – CrossAreaCancelMsg<br>Interface of cancellation of application for leaving traffic control area – CrossAreaCancelMsg |
| **Approval for entering/leaving traffic control area asynchronously** | The host system does not perform traffic control. | Interface of approval for entering traffic control area asynchronously – CrossAreaApproveMsg<br>Interface of approval for leaving traffic control area asynchronously - CrossAreaApproveMsg |
| **Passing status feedback** | The host system does not perform traffic control. | Interface of entering status feedback - AreaMovingPhaseMsg<br>Interface of leaving status feedback - AreaMovingPhaseMsg |
| **Traffic area locking/releasing** | The host system does not perform traffic control. | Traffic area locking interface - TrafficAreaLockMsg<br>Traffic area release interface - TrafficAreaLockMsg |

This document shall not be disseminated in any form without the written permission of Geek+.

| Business scenario | Service conditions | Service interface |
|---|---|---|
| **Robot information** | | Robot information query interface - RobotInfoMsg |
| **Charging station information** | | Charging station information query interface - ChargerInfoMsg |
| **Map information** | | Map information query interface - MapInfoMsg |
| **Custom page** | Workstation operation customization page | Station list query interface - StationListMsg<br>Area query interface - AreaListMsg<br>Location query interface - LocationListMsg<br>Workflow list query interface - WorkflowListMsg<br>Workflow start interface - WorkflowStartMsg<br>Workflow next-step interface - InstanceOperationMsg<br>Workflow cancellation interface - InstanceOperationMsg<br>Workflow priority adjustment interface - InstanceOperationMsg<br>Task callback interface - MovingCallbackMsg |

# 5 Interface Logs

Users can query interface communication records and detailed messages on the Log Management > Interface Log page. Query interfaces do not display response messages.

This document shall not be disseminated in any form without the written permission of Geek+.

# 6 Interface Description

## 6.1 Task Interfaces

### 6.1.1 Point-to-point Moving Interface - MovingRequestMsg

This interface is used for moving containers (shelves) between two points or no-load robot operations between two points.

Scenario:

1. If the host system manages shelves and a designated shelf needs to be moved, the host system can pass the shelf number and destination location code for GMS to move the shelf to the designation location.
2. If the host system does not manage shelves but manages shelf storage locations, the host system can pass the start location code and destination location code for GMS to move the shelf from the start location to the destination location.
3. If the host system manages locations or stations and wishes the no-load robot to move from one location to the other, it can pass the start location code and destination location code.

Usage method:

1. Configure robot interaction at nodes in GMS to allow GMS to generate the workflow according to the locations passed from the host system and implement the task.
2. Create the workflow in GMS in advance to allow GMS to match the exiting workflow according to the locations passed from the host system and implement the workflow.

**Request message**

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be MovingRequestMsg. | |
| **taskType** | string | No | Matches the workflow template code in the workflow template. | If this field is empty, use the default system template corresponding to the message type. |
| **taskCode** | string | No | Host task code/bill number, which will be returned to the upstream customer during response or call-back. | |
| **scanCode** | string | No | Pallet code transmitted by the upstream system | Functionality:<br>This field is used for subsequent code verification and comparison. |
| **locationFrom** | string | No | Start location code<br>If there is only one location in the workstation (station), the workstation ID can be passed instead.<br>Matching sequence: | **Choose any of them. If two items are passed, the containerCode shall prevail.** |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | **map node > workstation > area** | |
| **containerCode** | string | No | Container ID | |
| **containerCategory** | string | No | Container type code | Usage scenario: If the start location is an **area** with more than two types of containers, pass this parameter when originating a task. If there is only one type of container in the area as no container type or more than one container type is configured in the area, but no container type is passed when the container is fetched, this type of container will be automatically allocated. |
| **locationTo** | string | Yes | Destination location code<br>If there is only one location in the workstation (station), the workstation ID | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | can be passed instead.<br>Matching sequence: **map node > workstation > area** | |
| **priority** | integer | No | Priority of task implementation: The smaller the number, the higher the priority. | If this field is not entered, use the priority in the template; if it is entered, use the entered priority. |
| **needTime** | string | No | Required task completion time | This field affects the task priority. Under the same priority, the smaller the needTime (format: **yyyy-MM-dd HH:mm:ss**), the higher the priority. |
| **flowRuleCondition** | array | No | There are 20 extra parameters:<br>flowRuleCondition[<br>"extraParam1":"",<br>... ...<br>"extraParam20":"",<br>] | |

Example of request message:

```
{
    "header": {
        "requestId": "b4134151-81c9-4ab0-808c-be20ffc1d074",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2022-10-21 16:20:24"
    },
    "body": {
        "msgType": "MovingRequestMsg",
        "taskType": "M000163",
        "taskCode": "",
        "locationFrom": "4020",
        "containerCode": "",
        "containerCategory": "",
        "locationTo": "4016",
        "priority": 1,
        "flowRuleCondition":[
            "extraParam1":"",
            ... ...
            "extraParam20":"",
        ]
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |

| Field | Type | Description | Remarks |
|---|---|---|---|
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |
| **instancePriority** | integer | Priority of workflow instance (task): The smaller the value, the higher the priority. 1 represents the highest priority. | |
| **containerCode** | string | Container ID | |

Example of response message


```
{
    "header": {
        "responseId": "b4134151-81c9-4ab0-808c-be20ffc1d074",
        "msgType": "MovingRequestMsg",
        "code": "0",
        "requestTime": "2022-10-21 16:20:24",
        "clientCode": "geekplus",
        "msg": "调用成功"
    },
    "body": {
        "workflowCode": "F202211020000007",
        "taskCode": "",
        "instanceId": 238,
        "instancePriority": 1,
        "containerCode": "00000112"
    }
}
```


This document shall not be disseminated in any form without the written permission of Geek+.

### 6.1.2 Interface of Moving between Dynamic Points - DynamicMovingRequestMsg

The task workflow/step is certain, but the station point of each step is uncertain. The workflow in progress shall be completed or a new workflow shall be originated according to the start location ID and destination location ID passed for a single step.

Usage method:

Configure the robot interaction template in GMS to allow GMS to match the workflow instance in progress according to the passed locations to proceed with the current workflow or generate the workflow and implement the task.

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be DynamicMovingRequestMsg. | |
| **taskType** | string | No | Matches the workflow template code in the workflow template. | If this field is empty, use the default system template corresponding to the message type. |
| **taskCode** | string | No | Host task code/bill number, which will be returned to the upstream customer during response or call-back. | **Choose any of them. If both items are passed, the taskCode shall prevail. If neither items is passed, create a new** |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **instanceId** | long | No | Workflow instance (task) ID | **workflow instance during task implementation.** |
| **scanCode** | string | No | Pallet code transmitted by the upstream system | Functionality:<br>1) After this interface is called, the container at an empty start point is automatically entered, but if the same container already exists at the start point, the container will not enter.<br>2) This field is used for subsequent code verification and comparison. |
| **locationFrom** | string | No | Start location code<br>If there is only one location in the workstation (station), the workstation ID can be passed instead.<br>Matching sequence:<br>**map node > workstation > area** | **Choose any of them. If two items are passed, the containerCode shall prevail.** |
| **containerCode** | string | No | Container ID | |
| **containerCategory** | string | No | Container type code | Usage scenario: If the |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | start location is an **area** with more than two types of containers, pass this parameter when originating a task.<br>If there is only one type of container in the area as no container type or more than one container type is configured in the area, but no container type is passed when the container is fetched, this type of container will be automatically allocated. |
| **locationTo** | string | Yes | Destination location code<br>If there is only one location in the workstation (station), the workstation ID can be passed instead.<br>Matching sequence: **map node > workstation > area** | |
| **priority** | integer | No | Priority of task implementation: The | If this field is not entered, use the priority in the |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | smaller the number, the higher the priority. | template; if it is entered, use the entered priority. |
| **needTime** | string | No | Required task completion time | This field affects the task priority. Under the same priority, the smaller the needTime (format: **yyyy-MM-dd HH:mm:ss**), the higher the priority. |
| **extraParams** | array | No | There are 20 extra parameters:<br>extraParams[<br>"extraParam1":"",<br>... ...<br>"extraParam20":"",<br>] | |

Example of request message:


```
{
    "header": {
        "requestId": "fc12722e-1b2e-41d3-98cf-5759c549374f",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "DynamicMovingRequestMsg",
```

```
            "taskType": "M000162",
            "taskCode": "11",
            "scanCode": "",
            "instanceId": "",
            "locationFrom": "4020",
            "containerCode": "",
            "containerCategory": "",
            "locationTo": "4016",
            "priority": 15,
            "needTime": "",
            "extraParams":[
                "extraParam1":"",
                ... ...
                "extraParam20":"",
            ]
        }
    }
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |
| **instancePriority** | integer | Priority of workflow instance (task): The smaller the value, the higher the priority. 1 | |

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | represents the highest priority. | |
| **containerCode** | string | Container ID | |
| **taskId** | long | Subtask ID | |
| **locationFrom** | string | Task start location code | |
| **locationTo** | string | Task destination location code | |

Example of a response message:


```
{
    "header": {
        "responseId": "c36ef883-abc5-4a82-956f-a55eb33d8a6d",
        "msgType": "DynamicMovingRequestMsg",
        "code": "0",
        "requestTime": "2020-06-06 18:03:24",
        "clientCode": "geekplus",
        "msg": "调用成功"
    },
    "body": {
        "workflowCode": "F202211020000005",
        "taskCode": "11",
        "instanceId": 236,
        "instancePriority": 1,
        "containerCode": "00000112",
        "taskId": 504,
        "locationFrom": "-1",
        "locationTo": "4020"
    }
}
```


This document shall not be disseminated in any form without the written permission of Geek+.

### 6.1.3 Interface of Moving from a Point to Multiple Points - MultiMovingRequestMsg

The robot starts from a location to multiple locations. It can go to destination locations by destination location IDs ranked in ascending order or by the sequence of location sending.

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be MultiMovingRequestMsg. | |
| **taskType** | string | No | Matches the workflow template code in the workflow template. | If this field is empty, use the default system template corresponding to the message type. |
| **taskCode** | string | No | Host task code/bill number, which will be returned to the upstream customer during response or call-back. | |
| **scanCode** | string | No | Pallet code transmitted by the upstream system | Functionality:<br>1) After this interface is called, the container at an empty start point is automatically entered, but if the same container already exists at the start |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | point, the container will not enter.<br>2) This field is used for subsequent code verification and comparison. |
| **locationFrom** | string | No | Start location code<br>If there is only one location in the workstation (station), the workstation ID can be passed instead.<br>Matching sequence: **map node > workstation > area** | **Choose any of them.**<br>**If two items are passed, the containerCode shall prevail.** |
| **containerCode** | string | No | Container ID | |
| **containerCategory** | string | No | Container type code | Usage scenario: If the start location is an **area** with more than two types of containers, pass this parameter when originating a task.<br>If there is only one type of container in the area as no container type or more than one container type is configured in the area, but |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | no container type is passed when the container is fetched, this type of container will be automatically allocated. |
| **flowStrategy** | integer | No | Flow strategy | This field is required to be entered when there are multiple destination locations:<br>1: by the passing sequence (default)<br>2: by code value ranked in ascending order |
| **dests** | List | Yes | Collection of destination locations | |
| **locationTo** | string | | Destination location code<br>If there is only one location in the workstation (station), the workstation ID can be passed instead.<br>Matching sequence: **map node > workstation > area** | |
| | | | | |
| **priority** | integer | No | Priority of task implementation: The smaller the number, the | If this field is not entered, use the priority in the template; if it is entered, |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | higher the priority. | use the entered priority. |
| **needTime** | string | No | Required task completion time | This field affects the task priority. Under the same priority, the smaller the needTime (format: **yyyy-MM-dd HH:mm:ss**), the higher the priority. |
| **extraParams** | array | No | There are 20 extra parameters:<br>extraParams[<br>"extraParam1":"",<br>... ...<br>"extraParam20":"",<br>] | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "MultiMovingRequestMsg",
        "taskType": "M000161",
```

```
            "taskCode": "11",
            "scanCode": "",
            "locationFrom": "35",
            "containerCode": "",
            "containerCategory": "",
            "flowStrategy": 2,
            "priority": 1,
            "needTime": "",
            "dests": [
                {
                    "locationTo": "52"
                },
                {
                    "locationTo": "30"
                }
            ],
            "extraParams":[
                "extraParam1":"",
                ... ...
                "extraParam20":"",
            ]
        }
    }
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |

| Field | Type | Description | Remarks |
|---|---|---|---|
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |
| **instancePriority** | integer | Priority of workflow instance (task): The smaller the value, the higher the priority. 1 represents the highest priority. | |
| **taskId** | long | Subtask ID | |
| **containerCode** | string | Container ID | |
| **locationFrom** | string | Task start location code | |
| **locationTo** | string | Task destination location code | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "msgType": "MultiMovingRequestMsg",
        "code": "0",
        "requestTime": "2020-06-06 18:03:24",
        "clientCode": "geekplus",
        "msg": "调用成功"
    },
    "body": {
        "workflowCode": "F202210310000005",
        "taskCode": "11",
        "instanceId": 215,
        "instancePriority": 1,
```

```
            "containerCode": "00000115",
            "taskId": 440,
            "locationFrom": "-1",
            "locationTo": "35"
        }
    }
```

### 6.1.4 Dynamic Unit Template Moving Interface - DynamicUnitMovingRequestMsg

**Tasks can be updated unlimitedly, and the workflow only ends after isEnd is passed.**

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be DynamicUnitMovingRequestMsg. | |
| **instruction** | string | Yes | Execution instruction | START - Create<br>NEXT_STEP - Update<br>BREAK - Break |
| **taskType** | string | No | Matches the workflow template code in the workflow template. | If this field is empty, use the default system template corresponding to the message type. |
| **taskCode** | string | No | Host task code/bill number, which will be returned to the upstream | **Choose any of them to be passed.** |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | customer during response or call-back. | **If multiple items are passed, take the first non-empty value passed in the following sequence as the condition: taskCode > instanceId** |
| **instanceId** | long | No | Workflow instance (task) ID | |
| **locationTo** | string | Yes | Destination location code<br>If there is only one location in the workstation (station), the workstation ID can be passed instead.<br>Matching sequence: map node > workstation > area | |
| **toBinOrder** | string | No | Bin order ID at destination stock bin | |
| **toBinType** | integer | No | Stock bin type at destination location, numbered 0,1,2,3... 0 by default.<br>Passed values other than 0 shall be determined according to the project. | |
| **containerCategory** | string | No | Container type code<br>Usage scenario: If the first destination node (first node of the workflow) is an area with more | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | than two types of containers, pass a container type when originating a task. | |
| **robotProduct** | string | No | Designated robot model. If the robot ID is designated at the same time, the robot ID shall prevail. | |
| **robotId** | List of integer | No | ID of a robot arriving at the destination location to implement the workflow task. If a robot is to be designated to implement a task, this field must be passed. If the robot type and robot ID are configured in the template, the passed field shall prevail. | |
| **robotRule** | string | No | If the robot ID is designated, this field is required and must be implemented.<br>Otherwise, it does not need to be passed, given precedence by default. | 0 - precedence<br>1 - force |
| **robotSide** | string | No | Robot side orientation at destination location<br>The robot side direction takes effect when assigned for the interface during interactive configuration. | F (front)<br>B (back)<br>L (left)<br>R (right) |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **isEnd** | integer | No | If the destination location is the end point of the workflow, a complete workflow is formed upon the creation of the workflow and ends after being implemented. | 1 - destination node (namely end point); 0 - non-destination node, 0 by default if this field is not passed; |
| **priority** | integer | No | Priority of task implementation: The smaller the number, the higher the priority. If this field is not entered, use the priority in the template; if it is entered, use the entered priority. | |
| **extraParams** | array | No | There are 20 extra parameters, which shall be defined according to the business conditions. One parameter or multiple parameters can be passed.<br>extraParams[<br>"extraParam1":"",<br>... ...<br>"extraParam20":"",<br>] | |

Example of request message:

```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
```

This document shall not be disseminated in any form without the written permission of Geek+.

```
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "DynamicUnitMovingRequestMsg",
        "instruction": "START",
        "taskType": "M000161",
        "taskCode": "11",
        "instanceId": "",
        "locationTo": "4020",
        "toBinOrder": "",
        "toBinType": "",
        "containerCategory": "",
        "robotProduct": "",
        "robotId": [],
        "robotRule": "",
        "robotSide": "",
        "isEnd": 0,
        "priority": 1,
        "deviation": "",
        "extraParams": [
            "extraParam1": "",
            ... ...
            "extraParam3": ""
        ]
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |
| **instancePriority** | integer | Priority of workflow instance (task): The smaller the value, the higher the priority. 1 represents the highest priority. | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "msgType": "DynamicUnitMovingRequestMsg",
        "code": "0",
        "requestTime": "2020-06-06 18:03:24",
        "clientCode": "geekplus",
        "msg": "调用成功"
    },
    "body": {
        "workflowCode": "F202210310000005",
        "taskCode": "11",
        "instanceId": 215,
        "instancePriority": 1
    }
}
```


This document shall not be disseminated in any form without the written permission of Geek+.

## 6.2 Task Flow (Workflow) Interfaces

### 6.2.1 Task Flow (Workflow) Start Interface - WorkflowStartMsg

The host system starts a preset workflow of moving between multiple points, which could consist of multiple nodes. After the business operation of a node is performed, continue to implement the workflow in the next node according to the returned task code or current node ID.

Detailed usage method:

| Scenario | workflowCode | startPoint | needPoint | nextPoint | locationCode | containerCategory | containerCode | flowRuleCondition |
|---|---|---|---|---|---|---|---|---|
| **Single workflow**<br>**Single start location**<br>**Start point - location** | Choose one from the three. | | | | | | | |
| **Single workflow**<br>**Single start location**<br>**Start point - workstation with only one location** | Choose one from the three. | | | | | | | |
| **Single workflow**<br>**Single start location**<br>**Start point - workstation with multiple locations** | Choose one from the three. | | | | Required | | | |

| Scenario | workflowCode | startPoint | needPoint | nextPoint | locationCode | containerCategory | containerCode | flowRuleCondition |
|---|---|---|---|---|---|---|---|---|
| **Single workflow Single start point (with the same logic as the above) Multiple branches in the next node and automatic system judgment according to the flow rule** | Choose one from the three. | | | | Required | | | Required |
| **Single workflow Single start point (with the same logic as the above) Multiple branches in the next node and manual selection** | Choose one from the three. | | | Required | Required | | | |
| **Single workflow Single start location Start point -area** | Required | | | | | Pick one from the two or leave the field empty | | |
| **Single workflow Single start location Start point - virtual node** | Pick one from the two | | Pick one from the two | | | | Required | |

| Scenario | workflowCode | startPoint | needPoint | nextPoint | locationCode | containerCategory | containerCode | flowRuleCondition |
|---|---|---|---|---|---|---|---|---|
| **Single workflow**<br>**Single start location**<br>**Start point -area**<br>**Need point - workstation** | Pick one from the two | | Pick one from the two | | | Pick one from the two<br>or leave the field empty | | |

## Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be WorkflowStartMsg. | |
| **taskCode** | string | No | Host task code/bill number, which will be returned to the upstream customer during response or call-back. | |
| **workflowCode** | string | No | Workflow code | |
| **startPoint** | string | No | Start point code | Start the workflow based on the passed points that can be any nodes in the map, workstation, or area. The |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | system will match the workflow according to the passed information. However, if multiple workflows are matched, an error message will be returned. |
| **locationCode** | string | No | Location code | |
| **needPoint** | string | No | Need point | It means any point other than the start point. The distribution mode of the workstation shall be judged: The workflow can be started for material calling; proactive distribution can only occur at the start point of the workflow. |
| **containerCategory** | string | No | Container type code | |
| **containerCode** | string | No | Container ID | Verification rules:<br>1. The passed container ID shall prevail, and the container type field becomes invalid.<br>2. Check whether the container matches the |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | started workflow by the container ID: The area-point workflow will verify whether the container belongs to the area; the point-to-point/workstation-point workflow will verify whether the container is located at the start point of the workflow. |
| **nextPoint** | string | No | Next node. This field is used when there are multiple next nodes (with branches) and manual designation is required. | |
| **flowRuleCondition** | object | No | Branch flow condition. There are 20 conditions, and one or multiple conditions can be passed. When a robot can move from a start point to multiple destination locations, this item can be passed, and the system will judge the destination location of the robot accordingly.<br>For example:<br>When extraParam1 is 1, move from A to B.<br>When extraParam2 is 2, move from A to C. | |

**PS: For the "required" fields in the above request message body, values**

This document shall not be disseminated in any form without the written permission of Geek+.

**that must be passed shall be determined with reference to the specific business scenario.**

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "WorkflowStartMsg",
        "taskCode": "",
        "workflowCode": "F000021",
        "startPoint": "",
        "locationCode": "",
        "containerCategory": "",
        "containerCode": "",
        "nextPoint": "",
        "flowRuleCondition": [
            "extraParam1": "",
            ... ...
            "extraParam20": ""
        ]
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |
| **instancePriority** | integer | Priority of workflow instance (task): The smaller the value, the higher the priority. 1 represents the highest priority. | |
| **taskId** | long | Subtask ID | |
| **containerCode** | string | Container ID | |
| **locationFrom** | string | Task start location code | |
| **locationTo** | string | Task destination location code | |
| **areaFrom** | string | Task start area code | |
| **areaTo** | string | Task destination area code | |
| **stationFrom** | string | Task start workstation code | |
| **stationTo** | string | Task destination workstation code | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "WorkflowStartMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
```

```
            "workflowCode": "",
            "taskCode": "",
            "instanceId": 8,
            "instancePriority": 1,
            "taskId": 1,
            "containerCode": "A000001",
            "locationFrom": "00250335",
            "locationTo": "00250336",
            "areaFrom": "",
            "areaTo": "",
            "stationFrom": "",
            "stationTo": ""
        }
    }
```


### 6.2.2 Task (Workflow) Next-step Interface - InstanceOperationMsg - NEXT_STEP

When a robot is to go to multiple destination locations, after the business operation is performed at a location, this interface is used to trigger it to go to the next location.

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be InstanceOperationMsg. | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **instruction** | string | Yes | Execution instruction: The value must be NEXT_STEP. | |
| **instanceId** | long | No | Workflow instance (task) ID | **Choose any of them to be passed. If multiple items are passed, take the first non-empty value passed in the following sequence as the condition: instanceId > containerCode > locationCode > taskCode** |
| **containerCode** | string | No | Container ID, specifically the existing container ID in the workflow instance | |
| **locationCode** | string | No | Location code. If there is only one location in the workstation (station), the workstation ID can be passed instead. | |
| **taskCode** | string | No | Task code. A task code can be used when it only corresponds to a workflow instance in progress; if a task code corresponds to multiple workflow instances in progress, an error message will be returned. | |
| **needPoint** | string | No | Manually designated code of the next node | |
| **containerCategory** | string | No | Container type code | |
| **flowRuleCondition** | object | No | Branch flow condition. There are 20 conditions, and one or multiple conditions can be passed. When a robot can move from a start point to multiple | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | destination locations, this item can be passed, and the system will judge the destination location of the robot accordingly.<br>For example:<br>When extraParam1 is 1, move from A to B.<br>When extraParam2 is 2, move from A to C. | |
| | | | | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "InstanceOperationMsg",
        "instruction": "NEXT_STEP",
        "instanceId": "2",
        "containerCode": "",
        "locationCode": "",
        "taskCode": "",
        "nextPoint": "",
        "flowRuleCondition": [
            "extraParam1": "",
            ... ...
            "extraParam20": ""
```

```
            ]
        }
    }
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |
| **instancePriority** | integer | Priority of workflow instance (task): The smaller the value, the higher the priority. 1 represents the highest priority. | |
| **taskId** | long | Subtask ID | |
| **containerCode** | string | Container ID | |
| **locationFrom** | string | Task start location code | |
| **locationTo** | string | Task destination location code | |
| **areaFrom** | string | Task start area code | |
| **areaTo** | string | Task destination area code | |
| **stationFrom** | string | Task start workstation code | |
| **stationTo** | string | Task destination workstation code | |

Example of a response message:

```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
```

```
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "InstanceOperationMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "workflowCode": "",
        "taskCode": "",
        "instanceId": 8,
        "taskId": 1,
        "containerCode": "A000001",
        "locationFrom": "00250335",
        "locationTo": "00250336",
        "areaFrom": "",
        "areaTo": "",
        "stationFrom": "",
        "stationTo": ""
    }
}
```


## 6.2.3 Workflow Query Interface - WorkflowListMsg

### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be WorkflowListMsg. | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **workflowName** | string | No | Workflow name | **Optional.**<br>**If no field is filled, the system queries all. Relations among entered conditions are "AND".** |
| **workflowCode** | string | No | Workflow code | |
| **priority** | integer | No | Priority: The smaller the number, the higher the priority. | |
| **Purpose** | integer | No | Robot usage<br>1: lift<br>2: roller<br>4: forklift<br>8: composite | |
| **robots** | List of integer | No | List of robot IDs | |
| **status** | integer | No | Workflow status<br>1: published<br>2: under design | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01"
    },
    "body": {
        "msgType": "WorkflowListMsg",
        "workflowName": "",
        "workflowCode": "",
        "priority": "",
```

```
            "purpose": "",
            "robots": [],
            "status": ""
        }
    }
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| | List | List of workflows | |
| **workflowName** | string | Workflow name | |
| **workflowCode** | string | Workflow code | |
| **priority** | integer | Priority: The smaller the number, the higher the priority. | |
| **Purpose** | integer | Robot usage | 1: lift<br>2: roller<br>4: forklift<br>8: composite |
| **robots** | List of integer | List of robot IDs | |
| **status** | integer | Workflow status | 1: published<br>2: under design |
| **nodes** | List | Workflow node details | |
| **nodeParentCode** | string | Parent node code | |
| **nodeParentName** | string | Parent node name | |
| **nodeParentType** | integer | Parent node type | 0: start |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | 2: workstation<br>4: area<br>7: virtual node<br>10: map node |
| **nodeCode** | string | Node ID | |
| **nodeName** | string | Node name | |
| **nodeType** | integer | Node type | 0: start<br>2: workstation<br>4: area<br>7: virtual node<br>10: map node |
| **priority** | integer | Node priority | |
| **flowRule** | string | flow rule | It refers to the conditions for flowing to this node, represented by a specific expression. |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "msgType": "WorkflowListMsg",
        "code": "0",
        "msg": "success"
    },
    "body": [
        {
```

```
            "workflowName": "",
            "workflowCode": "",
            "operationMode": "",
            "priority": "",
            "purpose": "",
            "robots": [],
            "status": "",
            "nodes": [
                {
                    "nodeParent": "",
                    "nodeCode": "023021",
                    "nodeName": "门板组装",
                    "priority": "1",
                    "flowRule": "完成情况==正常完成 & extraParam1==1"
                },
                {
                    "nodeParent": "1",
                    "nodeCode": "023022",
                    "nodeName": "门板加工",
                    "priority": "1",
                    "flowRule": "完成情况==正常完成"
                }
            ]
        }
    ]
}
```

### 6.2.4 Workflow Instance (task) Query Interface - WorkflowInstanceListMsg

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be WorkflowInstanceList Msg. | |
| **workflowName** | string | No | Workflow name | **Optional. If no field is filled, the system queries all. Relations among entered conditions are "AND".** |
| **workflowCode** | string | No | Workflow code | |
| **instanceId** | long | No | Workflow instance ID | |
| **taskCode** | string | No | Host task code/bill number | |
| **instancePriority** | integer | No | Workflow instance priority | |
| **instanceStatus** | integer | No | Workflow instance status<br>10: create<br>20: Being implemented<br>30: completed<br>31: exception/closed | |
| **nodeCode** | string | No | Current location or destination location of workflow instance | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **containerCode** | string | No | Container ID currently used in the workflow | |
| **robot** | integer | No | Robot ID currently used in the workflow | |
| **startTime** | string | No | Workflow start time (start point of period) | |
| **endTime** | string | No | Workflow end time (end point of period) | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01"
    },
    "body": {
        "msgType": "WorkflowInstanceListMsg",
        "workflowName": "",
        "workflowCode": "",
        "instanceId": "",
        "taskCode": "",
        "instancePriority": "",
        "instanceStatus": "",
        "nodeCode": "",
        "containerCode": "",
        "robot": "",
        "startTime": "",
```

```
            "endTime": ""
        }
    }
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| | List | List of workflow instances | |
| **workflowName** | string | Workflow name | |
| **workflowCode** | string | Workflow code | |
| **instanceId** | long | Workflow instance ID | |
| **taskId** | long | Subtask ID | |
| **taskCode** | string | Host task code/bill number | |
| **instancePriority** | integer | Workflow instance priority | |
| **instanceStatus** | integer | Workflow instance status | 10: create<br>20: Being implemented<br>30: completed<br>31: exception/closed |
| **nodeCode** | string | Current location or destination location of workflow instance | |
| **nodeName** | string | Current location of workflow instance or task segment currently implemented in the workflow, such as | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | location A - location B | |
| **locationFrom** | string | Task start location code | |
| **locationTo** | string | Task destination location code | |
| **containerCode** | string | Container ID currently used in the workflow | |
| **robot** | integer | Robot ID currently used in the workflow | |
| **startTime** | string | Workflow start time | |
| **taskStatus** | integer | Task phase | 10: create;<br>15: ready to send;<br>20: being implemented;<br>25: waiting at node;<br>30: done |
| **robotStatus** | string | Robot task phase | GO_FETCHING: The robot is fetching the shelf (container)<br>SHELF_FETCHED: The robot has fetched the shelf.<br>GO_DELIVERING: The robot is moving the shelf.<br>QUEUING: The robot is queuing up.<br>SHELF_ARRIVED: The shelf has arrived at the destination.<br>GO_RETURN: The robot is returning the shelf.<br>SHELF_TURNING: The shelf |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | is being rotated.<br>MOVING: The robot is moving.<br>ARRIVED_WAIT_POINT: The robot has arrived at the wait point.<br>LEAVING_WAIT_POINT: The robot has left the wait point.<br>RECEIVE_FINISH: The roller robot has received the goods.<br>DROP_FINISH: The roller robot has dropped the goods.<br>ARRIVED_ELEVATOR_ENTRY: The robot has arrived at the elevator, waiting for entering the elevator.<br>ENTERED_ELEVATOR: The robot has stepped into the elevator.<br>LEAVED_ELEVATOR: The robot has left the elevator<br>ARRIVED: The robot has arrived at the destination.<br>FETCHED: The task has already been completed.<br>TURN_FINISH: Turning has finished. |
| **robotError** | string | Robot task error | For details of robot task error, |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | see: 8.3 Robot Exception Codes |

Example of a response message:


```
{
    "header": {
        "responseId": "74347428-ac28-47b1-ba21-8c7a9c39f48c",
        "msgType": "WorkflowInstanceListMsg",
        "code": "0",
        "requestTime": null,
        "clientCode": null,
        "msg": "调用成功"
    },
    "body": [
        {
            "workflowName": "4 楼等待点测试",
            "workflowCode": "F000064",
            "instanceId": 239,
            "taskId": 515,
            "taskCode": "",
            "instancePriority": 1,
            "instanceStatus": 20,
            "nodeCode": null,
            "nodeName": null,
            "locationFrom": "-1",
            "locationTo": "4020",
            "containerCode": "00000112",
            "robot": "1100094",
```

"startTime": "2022-11-03 10:28:15",
"taskStatus": 30,
"robotStatus": "ARRIVED",
"robotError": null
},
{
"workflowName": "4 楼等待点测试",
"workflowCode": "F000064",
"instanceId": 239,
"taskId": 516,
"taskCode": "",
"instancePriority": 1,
"instanceStatus": 20,
"nodeCode": null,
"nodeName": null,
"locationFrom": "4020",
"locationTo": "4016",
"containerCode": "00000112",
"robot": "1100094",
"startTime": "2022-11-03 10:28:15",
"taskStatus": 30,
"robotStatus": "SHELF_ARRIVED",
"robotError": null
},
{
"workflowName": "4 楼等待点测试",
"workflowCode": "F000064",
"instanceId": 239,

```
            "taskId": 517,
            "taskCode": "",
            "instancePriority": 1,
            "instanceStatus": 20,
            "nodeCode": null,
            "nodeName": null,
            "locationFrom": "4016",
            "locationTo": "4002",
            "containerCode": "",
            "robot": "1100094",
            "startTime": "2022-11-03 10:28:15",
            "taskStatus": 20,
            "robotStatus": "ARRIVED_WAIT_POINT",
            "robotError": null
        }
    ]
}
```


### 6.2.5 Forced Task Flow Deletion Interface (Workflow Cancellation) - InstanceOperationMsg - WORKFLOW_DELETE

If the task flow is deleted by force, the task might not form a closed loop. For example, if the task is canceled, the loaded robot is unaware of where the shelf or pallet shall be unloaded. At this time, manual access of the task flow is required. Therefore, except in special circumstances, it is not recommended to use this interface.

**Note: By using this interface, the task flow can be deleted for the loaded**

**robot by force, but the loaded container shall be manually reset.**

## Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be InstanceOperationMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be WORKFLOW_DELETE. | |
| **instanceId** | long | No | Workflow instance (task) ID | **Choose any of them to be passed. If multiple items are passed, take the first non-empty value passed in the following sequence as the condition: instanceId > containerCode > locationCode > taskCode** |
| **containerCode** | string | No | Container ID, specifically the existing container ID in the workflow instance | |
| **locationCode** | string | No | Location code. If there is only one location in the workstation (station), the workstation ID can be passed instead. | |
| **taskCode** | string | No | Task code. A task code can be used when it only corresponds to a workflow instance in progress; if a task code corresponds to multiple workflow instances in | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | progress, an error message will be returned. | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "InstanceOperationMsg",
        "instruction": "WORKFLOW_DELETE",
        "instanceId": "2",
        "containerCode": "",
        "locationCode": "",
        "taskCode": ""
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |

Example of a response message:

```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "InstanceOperationMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "workflowCode": "",
        "taskCode": "",
        "instanceId": 8
    }
}
```


### 6.2.6 Interface of Task Flow Cancellation and Destination Change (Workflow Cancellation and Delivery of Goods to the Designated Point) - InstanceOperationMsg - WORKFLOW_DROP

Task flow cancellation and destination change are bound to call back the task. When the robot approaches the end point, it is unable to cancel the task, but the system will automatically create a task to instruct the robot to place the shelf at the location designated by the upstream system.

**Note: To call this interface, the recycling area shall be configured in the workflow instance or template, including the start point of workflow,**

**designated recycling area, and custom recycling area:**

**Start point of workflow: When configuring this type of recycling area, destLocationCode is an optional parameter. When this interface is called, the robot will place the shelf at the start point of workflow, usually the shelf lifting location.**

**Designated recycling area: When configuring this type of recycling area, a location can be designated as the recycling area, and destLocationCode is an optional parameter. When this interface is called, the robot will place the shelf at the designated location.**

**Custom recycling area: When configuring this type of recycling area, destLocationCode is a required parameter. When this interface is called, the system will instruct the robot to place the shelf at the location designated by this parameter.**

## Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be InstanceOperationMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be WORKFLOW_DROP. | |
| **instanceId** | long | No | Workflow instance (task) ID | **Choose any of them to be passed. If multiple items are passed, take** |
| **containerCode** | string | No | Container ID, specifically the existing container ID in the | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | workflow instance | **the first non-empty value passed in the following sequence as the condition:**<br><br>**instanceId > containerCode > locationCode > taskCode** |
| **locationCode** | string | No | Location code. If there is only one location in the workstation (station), the workstation ID can be passed instead. | |
| **taskCode** | string | No | Task code. A task code can be used when it only corresponds to a workflow instance in progress; if a task code corresponds to multiple workflow instances in progress, an error message will be returned. | |
| **destLocationCode** | string | No | Destination location code. If there is only one location in the workstation (station), the workstation ID can be passed instead. Matching sequence: **map node > workstation > area** | The destination location code takes effect only if **the custom recycling area is configured with higher priority**. |

Example of request message:

```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "InstanceOperationMsg",
        "instruction": "WORKFLOW_DROP",
        "instanceId": "2",
        "containerCode": "",
        "locationCode": "",
        "taskCode": "",
        "destLocationCode": "QY01"
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |

Example of a response message:


```
{
    "header": {
```

```
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "InstanceOperationMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "workFlowCode": "",
        "taskCode": "",
        "instanceId": 8
    }
}
```


## 6.2.7 Interface for Proceeding to Next Step of Task at Wait Point - InstanceOperationMsg - CLEAR_WAIT

### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be InstanceOperationMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be CLEAR_WAIT. | |
| **instanceId** | long | No | Workflow instance | **Choose any of** |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | (task) ID | **them to be passed. If multiple items are passed, take the first non-empty value passed in the following sequence as the condition: instanceId > taskCode > containerCode > robotTaskId > robot > waitLocation** |
| **taskCode** | string | No | Task code. A task code can be used when it only corresponds to a workflow instance in progress; if a task code corresponds to multiple workflow instances in progress, an error message will be returned. | |
| **containerCode** | string | No | Container ID, specifically the existing container ID in the workflow instance | |
| **robotTaskId** | long | No | Robot task ID | |
| **robot** | integer | No | Robot ID | |
| **waitLocation** | string | No | Wait point ID | |

Example of request message:

```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
```

```
    "body": {
        "msgType": "InstanceOperationMsg",
        "instruction": "CLEAR_WAIT",
        "instanceId": "2",
        "taskCode": "",
        "containerCode": "",
        "robotTaskId": "",
        "robot": "",
        "waitLocation": ""
    }
}
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |

Example of a response message:

```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "InstanceOperationMsg",
        "code": "0",
        "msg": "success"
```

This document shall not be disseminated in any form without the written permission of Geek+.

```
        },
        "body": {
            "workflowCode": "",
            "taskCode": "",
            "instanceId": ""
        }
    }
```

## 6.2.8 Task Flow (Workflow) Priority Adjustment Interface - InstanceOperationMsg - PRIORITY_ADJUSTMENT

### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be InstanceOperationMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be PRIORITY_ADJUSTMENT. | |
| **instanceId** | long | No | Workflow instance (task) ID | **Choose any of them to be passed. If multiple items are passed, take** |
| **containerCode** | string | No | Container ID, specifically the existing container ID in the workflow instance | |
| **locationCode** | string | No | Location code. If there is | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | only one location in the workstation (station), the workstation ID can be passed instead. | **the first non-empty value passed in the following sequence as the condition:**<br><br>**instanceId > containerCode > locationCode > taskCode** |
| **taskCode** | string | No | Task code. A task code can be used when it only corresponds to a workflow instance in progress; if a task code corresponds to multiple workflow instances in progress, an error message will be returned. | |
| **needTime** | string | No | Required task completion time in the format of yyyy-MM-dd HH:mm:ss | **Choose any of them to be passed.**<br><br>**If all fields are passed, the field ranking in the frontmost takes effect.** |
| **prioritySet** | string | No | Priority adjustment mode:<br>t: Adjust to the highest priority<br>b: Adjust to the lowest priority<br>u: Adjust to the immediately higher priority<br>d: Adjust to the immediately lower priority | |

Example of request message:

```
{
```

This document shall not be disseminated in any form without the written permission of Geek+.

```
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "InstanceOperationMsg",
        "instruction": "PRIORITY_ADJUSTMENT",
        "needTime": "",
        "prioritySet": "t",
        "instanceId": "2",
        "containerCode": "",
        "locationCode": "",
        "taskCode": ""
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |
| **instancePriority** | integer | Priority of workflow instance: The smaller the value, the higher the priority. 1 | |

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | represents the highest priority. | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "InstanceOperationMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "workflowCode": "",
        "taskCode": "",
        "instanceId": 8,
        "instancePriority": 1
    }
}
```


## 6.2.9 Container Turning Interface - InstanceOperationMsg - GO_TURN

When the robot arrives at the destination location before the whole task flow is completed, the component can rotate to achieve the effect of container turning.

**When the robot arrives at the destination location before the whole task flow is completed, the robot body can rotate.**

## Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be InstanceOperationMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be GO_TURN. | |
| **instanceId** | long | No | Workflow instance (task) ID | **Choose any of them to be passed. If multiple items are passed, take the first non-empty value passed in the following sequence as the condition: instanceId > containerCode > locationCode > taskCode** |
| **containerCode** | string | No | Container ID, specifically the existing container ID in the workflow instance | |
| **locationCode** | string | No | Location code. If there is only one location in the workstation (station), the workstation ID can be passed instead. | |
| **taskCode** | string | No | Task code. A task code can be used when it only corresponds to a workflow instance in progress; if a task code corresponds to multiple workflow instances in | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | progress, an error message will be returned. | |
| **neededSides** | string | No | Enter F, B, L, or R (namely front, back, left, and right) when the container rotates to the corresponding side. This field can be used only if the workstation orientation at the current location and the needed sides upon arrival are configured. | **Choose any of them to be passed. Either neededSides or turnAngle can be used, but not both at the same time.** |
| **turnAngle** | integer | No | Designated rotation angle, which can only be a multiple of 90° (ranging from -360° to 360°), such as 90°, 180°, 270°, 360°, and -90°, etc. This is the relative rotation angle. | |

Example of request message:

```
{
    "header": {
```

```
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "InstanceOperationMsg",
        "instruction": "GO_TURN",
        "instanceId": "2",
        "containerCode": "",
        "locationCode": "",
        "taskCode": "",
        "neededSides": "F",
        "turnAngle": ""
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |
| **containerCode** | string | Container ID | |
| **locationCode** | string | Location code | |

Example of a response message:


```
{
    "header": {
```

```
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "InstanceOperationMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "taskCode": "",
        "instanceId": 8,
        "containerCode": "A000001",
        "locationCode": "00250335"
    }
}
```


### 6.2.10 Robot Stop Interface - InstanceOperationMsg - STOP

Full-field emergency stop with no conditions passed stop the moving of the robot, but the actions of components on the robot body still continue. According to the passed conditions, separate emergency stop of a task will not only stop the robot from moving, but also stop the components of the robot body from operating (this function is only available for the roller robot for now).

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | value must be InstanceOperationMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be STOP. | |
| **instanceId** | long | No | Workflow instance (task) ID | **Choose any of them to be passed.**<br>**If multiple items are passed, take the first non-empty value passed in the following sequence as the condition:**<br><br>**instanceId > containerCode > locationCode > taskCode > robotTaskId > robot;**<br>**If no field is passed, the system will stop all.** |
| **containerCode** | string | No | Container ID, specifically the existing container ID in the workflow instance | |
| **locationCode** | string | No | Location code. If there is only one location in the workstation (station), the workstation ID can be passed instead. | |
| **taskCode** | string | No | Task code. A task code can be used when it only corresponds to a workflow instance in progress; if a task code corresponds to multiple workflow instances in progress, an error message will be returned. | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **robotTaskId** | long | No | Robot task ID | |
| **robot** | integer | No | Robot ID | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "InstanceOperationMsg",
        "instruction": "STOP",
        "instanceId": "2",
        "containerCode": "",
        "locationCode": "",
        "taskCode": "",
        "robotTaskId": "",
        "robot": ""
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |

| Field | Type | Description | Remarks |
|---|---|---|---|
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |
| **robotTaskId** | long | Robot task ID | |
| **robot** | integer | Robot ID | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "InstanceOperationMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "workflowCode": "",
        "taskCode": "",
        "instanceId": 8,
        "robotTaskId": "",
        "robot": ""
    }
}
```

### 6.2.11 Robot Recovery Interface - InstanceOperationMsg - ROBOT_RECOVER

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be InstanceOperationMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be ROBOT_RECOVER. | |
| **instanceId** | long | No | Workflow instance (task) ID | **Choose any of them to be passed. If multiple items are passed, take the first non-empty value passed in the following sequence as the condition: instanceId > containerCode >** |
| **containerCode** | string | No | Container ID, specifically the existing container ID in the workflow instance | |
| **locationCode** | string | No | Location code. If there is only one location in the workstation (station), the workstation ID can be passed instead. | |
| **taskCode** | string | No | Task code. A task code can be used when it | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | only corresponds to a workflow instance in progress; if a task code corresponds to multiple workflow instances in progress, an error message will be returned. | **locationCode > taskCode > robotTaskId > robot; If no field is passed, the system will recover all.** |
| **robotTaskId** | long | No | Robot task ID | |
| **robot** | integer | No | Robot ID | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "InstanceOperationMsg",
        "instruction": "ROBOT_RECOVER",
        "instanceId": "2",
        "containerCode": "",
        "locationCode": "",
        "taskCode": "",
        "robotTaskId": "",
```

```
            "robot": ""
        }
    }
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **workflowCode** | string | Workflow code | |
| **taskCode** | string | Returned host task code/bill number, etc. | |
| **instanceId** | long | Workflow instance (task) ID | |
| **robotTaskId** | long | Robot task ID | |
| **robot** | integer | Robot ID | |

Example of a response message:

```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "InstanceOperationMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "workflowCode": "",
        "taskCode": "",
        "instanceId": 8,
        "robotTaskId": "",
```

This document shall not be disseminated in any form without the written permission of Geek+.

```
            "robot": ""
        }
    }
```

## 6.2.12 Cell Locking/Unlocking Interface - InstanceOperationMsg - CELL_CODE

### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be InstanceOperationMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be CELL_CODE. | |
| **locationCode** | string | Yes | Destination location code | |
| **cellCodeType** | list of string | Yes | Locking and unlocking | LOCK: lock<br>UNLOCK: unlock |

Example of request message:

```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
```

```
    "body": {
        "msgType": "InstanceOperationMsg",
        "instruction": "CELL_CODE",
        "cellCodeType": "LOCK",
        "locationCode": "2111"
    }
}
```


## Response message

The response message body is of no significance.

Example of a response message:


```
{
    "header": {
        "responseId": "1a0f000d-cd2a-45db-9b4e-558a043dd743",
        "msgType": "InstanceOperationMsg",
        "code": "0",
        "requestTime": "2020-06-06 18:03:24",
        "clientCode": "geekplus",
        "msg": "调用成功"
    },
    "body": {
        "workflowCode": "",
        "taskCode": "",
        "instanceId": "",
        "taskId": "",
        "areaCodes": [],
        "classificationId": "",
        "extraParam2": "",
```

```
            "extraParam3": "",
            "extraParam4": "",
            "extraParam5": "",
            "extraParam6": "",
            "extraParam7": "",
            "extraParam8": "",
            "extraParam9": ""
        }
    }
```

## 6.3 Container (Shelf) Interfaces

### 6.3.1 Container (Shelf) Query Interface - ContainerListMsg

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be ContainerListMsg. | |
| **containerCode** | string | No | Container ID | **Optional.** **If no field is filled, the system queries all.** **Relations among entered conditions** |
| **containerCodes** | List of string | No | List of container IDs | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | are "OR". |

Example of request message:

```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01"
    },
    "body": {
        "msgType": "ContainerListMsg",
        "containerCode": "C000001",
        "containerCodes": []
    }
}
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| | List | List of containers | |
| **containerCode** | string | Container ID | |
| **containerCategory** | string | Container type | |
| **locationCode** | string | Container location code | |
| **areaCode** | string | Container area code | |
| **stationCode** | string | Container workstation code | |
| **status** | integer | Container status | 0: removed<br>1: added |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| **taskCode** | string | Host task code/bill number or workflow instance (task) ID of the container being operated | |

Example of a response message:


```
{
    "header": {
        "responseId": "9b40e9c6-3081-494e-a249-fa7debdf4995",
        "msgType": "ContainerListMsg",
        "code": "0",
        "requestTime": null,
        "clientCode": null,
        "msg": "调用成功"
    },
    "body": [
        {
            "containerCode": "00000112",
            "containerCategory": "S0000001",
            "locationCode": "4020",
            "areaCode": null,
            "stationCode": "S01",
            "status": 2,
            "taskCode": "234"
        }
    ]
}
```

## 6.3.2 New Container Interface - ContainerOperationMsg

### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be ContainerOperationMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be NEW_CONTAINER. | |
| **containerCategory** | string | No | Host container type code | **If there is only one type of container in the system, this item is optional; if there are more than two types of containers, this item is required.** |
| **isEmpty** | integer | Yes | Empty container status | 0 - empty<br>1 - loaded |
| **containerCodes** | list of string | No | List of container IDs | **Pick one from the two If multiple items are** |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **numberOfContainer** | integer | No | Number of new containers, which will be automatically generated by the container coding system | **passed, take the first non-empty value passed in the following sequence as the condition: containerCodes > numberOfContainer;** |
| **startContainerCode** | string | No | Start code of new container | If this field is entered, it takes effect when containers are generated according to the numberOfContainer. The rightmost digit or multiple digits of this code shall be number. The digits of generated container ID shall be consistent with those of this code. This code is the start code, and the subsequent code shall be added in sequence. If it already exists in the system, the existing code shall be skipped. |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | If the total number of the rightmost digits is less than the set number, the generated number shall prevail. If this field is left empty, the system will automatically assign a 8-digit number container ID. |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "ContainerOperationMsg",
        "instruction": "NEW_CONTAINER",
        "containerCategory": "",
        "isEmpty": "0",
        "containerCodes": [
            "A000001",
            "A000002"
        ],
```

```
            "numberOfContainer": "",
            "startContainerCode": ""
        }
    }
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **success** | list | | |
| **numberOfSuccess** | integer | Number of successfully added containers | |
| **containers** | list | List of successfully added containers | |
| **containerCategory** | string | Container type code or host container type code | |
| **containerCode** | string | Container ID | |
| | | | |
| **fail** | list | | |
| **numberOfFail** | integer | Number of containers failing to be added | |
| **containers** | list | List of containers failing to be added | |
| **containerCategory** | string | Container type code or host container type code | |
| **containerCode** | string | Container ID | |
| | | | |
| **detail** | string | Failure reason | These containerCodes already exist. Insufficient number of |

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | available containerCodes. |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "ContainerOperationMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "success": {
            "numberOfSuccess": "1",
            "containers": [
                {
                    "containerCategory": "P001",
                    "containerCode": "A000001"
                }
            ]
        },
        "fail": {
            "numberOfFail": "1",
            "containers": [
                {
                    "containerCategory": "P002",
                    "containerCode": "A000002"
```

```
                }
            ],
            "detail": "These containerCodes already exsit."
        }
    }
}
```

## 6.3.3 Container (Shelf) Addition Interface - ContainerOperationMsg

### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be ContainerOperation Msg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be ADD_CONTAINER. | |
| **locationCode** | string | Yes | Destination location code of added container in GMS or in the host system If there is only one location in the | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | workstation (station), the workstation ID can be passed instead. | |
| **containerCode** | string | No | Container ID | If this field is left empty, the system will automatically generate it. |
| **containerCategory** | string | No | Host container type code | **If there is only one type of container in the system, this item is optional; if there are more than two types of containers, this item is required.** |
| **containerAngle** | float | No | Initial angle of the shelf | For example 90°. If this field is left empty, it is 90° by default. |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
```

```
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "ContainerOperationMsg",
        "instruction": "ADD_CONTAINER",
        "locationCode": "0231234",
        "containerCode": "A000001",
        "containerCategory": ""
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **locationCode** | string | Destination location code of added container | |
| **containerCode** | string | Container ID | |
| **containerCategory** | string | Container type code or host container type code | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "ContainerOperationMsg",
        "code": "0",
```

```
            "msg": "success"
        },
        "body": {
            "locationCode": "0231234",
            "containerCode": "A000001",
            "containerCategory": "P001"
        }
    }
```

### 6.3.4 Container (Shelf) Removal Interface - ContainerOperationMsg

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be ContainerOperationMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be REMOVE_CONTAINER. | |
| **locationCode** | string | Yes | Destination location code of removed container on GMS or the host system. If there is only one location in the | **Choose any of them. If two items are passed, the containerCode** |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | workstation (station), the workstation ID can be passed instead. | **shall prevail.** |
| **containerCode** | string | No | Container ID | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "ContainerOperationMsg",
        "instruction": "REMOVE_CONTAINER",
        "locationCode": "",
        "containerCode": "A000001"
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **locationCode** | string | Destination location code of removed container | |
| **containerCode** | string | Container ID | |
| **containerCategory** | string | Container type code or host container type | |

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | code | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "ContainerOperationMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "locationCode": "0231234",
        "containerCode": "A000001",
        "containerCategory": "P001"
    }
}
```


## 6.4 Workstation Interfaces

### 6.4.1 Interface for Starting Queuing for Receiving Goods - StationQueueMsg - START

There is the cycle time requirement for the delivery station. In the case of continuous delivery at a short time interval, the delivery station neither is able to call robots to receive goods in advance nor wants to wait for a long time after calling robots, which will affect the delivery of subsequent goods. In this

case, the function of station queuing for goods receiving can be started. After this function is started, the system will arrange robots to stand by near the station.

## Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be StationQueueMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be START. | |
| **stationCode** | string | Yes | Workstation code | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "StationQueueMsg",
        "instruction": "START",
        "stationCode": "2"
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **stationCode** | string | Workstation code | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "StationQueueMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "stationCode": "2"
    }
}
```


## 6.4.2 Interface for Stopping Queuing for Receiving Goods - StationQueueMsg - STOP

### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be StationQueueMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be STOP. | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **stationCode** | string | Yes | Workstation code | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "StationQueueMsg",
        "instruction": "STOP",
        "stationCode": "2"
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **stationCode** | string | Workstation code | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
```

```
            "msgType": "StationQueueMsg",
            "code": "0",
            "msg": "success"
        },
        "body": {
            "stationCode": "2"
        }
    }
```

### 6.4.3 Workstation Query Interface - StationListMsg

Query locations of the workstation (station) and robots thereunder.

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be StationListMsg. | |
| **stationName** | string | No | Workstation name | **Optional. If no field is filled, the system queries all. Relations among entered fields with the same meaning are "OR", while relations among entered fields** |
| **stationCode** | string | No | Workstation code | |
| **locationCode** | string | No | Location code | |
| **stationNames** | List of string | No | List of workstation names, for example ["a","b","c"] | |
| **stationCodes** | List of string | No | List of workstation codes, for example ["e","d","f"] | |
| **locationCodes** | List of string | No | List of location codes, through which the | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | workstation with the list of passed locations is queried | **with different meanings are "AND".** |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01"
    },
    "body": {
        "msgType": "StationListMsg",
        "stationName": "",
        "stationCode": "",
        "locationCode": "",
        "stationNames": [
            "a",
            "b",
            "c"
        ],
        "stationCodes": [],
        "locationCodes": []
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **stations** | List | List of workstations | |
| **stationName** | string | Workstation name | |
| **stationCode** | string | Workstation code | |
| **locations** | List | List of locations | |
| **locationCode** | string | Location code | |
| **containerCode** | string | Container ID | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "msgType": "StationListMsg",
        "code": "0",
        "msg": "success"
    },
    "body": [
        {
            "stationName": "1",
            "stationCode": "1",
            "locations": [
                {
                    "locationCode": "02250335",
                    "containerCode": "C000001"
                },
                {
                    "locationCode": "02250337",
                    "containerCode": ""
```

```
                }
            ]
        },
        {
            "stationName": "2",
            "stationCode": "2",
            "locations": []
        }
    ]
}
```

## 6.5 Map and Robot Resource Interfaces

### 6.5.1 Area Query Interface - AreaListMsg

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be AreaListMsg. | |
| **functionType** | List of string | No | Area function type<br>• Storage area: STORAGE<br>• Queuing area: QUEUING<br>• Traffic control area: TRAFFIC_CONTROL<br>• Emergency stop area: | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | STOP | |
| **areaName** | string | No | Area name | **Optional. If no field is filled, the system queries all. Relations among entered fields with the same meaning are "OR", while relations among entered fields with different meanings are "AND".** |
| **areaCode** | string | No | Area code | |
| **locationCode** | string | No | Location code | |
| **areaNames** | List of string | No | List of area names | |
| **areaCodes** | List of string | No | List of area codes | |
| **locationCodes** | List of string | No | List of location codes, through which the area with the list of passed locations is queried | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01"
    },
    "body": {
        "msgType": "AreaListMsg",
        "functionType": [
            "TRAFFIC_CONTROL"
```

```
            ],
            "areaName": "",
            "areaCode": "",
            "locationCode": "",
            "areaNames": [
                1,
                2,
                3
            ],
            "areaCodes": [
                "a",
                "b",
                "c"
            ],
            "locationCodes": []
        }
    }
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| | List | List of areas | |
| **areaName** | string | Area name | |
| **areaCode** | string | Area code | |
| **locations** | List | List of locations | |
| **locationCode** | string | Location code | |
| **containerCode** | string | Container ID | |

| Field | Type | Description | Remarks |
|---|---|---|---|
| **areaApexes** | object | Area apex information | |
| **mapId** | string | Map ID | Based on Geek+ map |
| **floorId** | string | Floor ID | Based on Geek+ map |
| **controlPoints** | List | List of apex coordinates | |
| **x** | string | X coordinate of location | Based on Geek+ map |
| **y** | string | Y coordinate of location | Based on Geek+ map |
| | | | |
| **functionParams** | List | List of area function parameters | |
| **functionType** | string | Area function type | Storage area: STORAGE<br>Queuing area: QUEUING<br>Traffic control area: TRAFFIC_CONTROL<br>Emergency stop area: STOP |
| **hitStrategy** | string | allocation strategy for containers in the storage area | 1 - default strategy<br>2 - first in first out<br>3 - first in last out<br>4 - minimum distance |
| **queuingStrategy** | string | Queuing strategy for robots in the queuing area | 1 - queuing in order<br>2 - queuing at the tail of the queue<br>3 - not queuing |
| **progressiveStrategy** | string | Progressive strategy for robots in the queuing area | 1 - making progress one by one<br>2 - filling up vacancy at the tail of the queue |
| **passStrategy** | string | Pass strategy for traffic | 10 - single manufacturer with |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | control area | a single entrance<br>20 - single manufacturer with multiple entrances<br>30 - multiple manufacturers with a single entrance |
| **entryStrategy** | string | Release strategy for traffic control area | 10 - first come first served<br>20 - first come first served (priority first)<br>30 - first occupied first served<br>40 - first occupied first served (priority first) |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011e33",
        "msgType": "AreaListMsg",
        "code": "0",
        "msg": "调用成功"
    },
    "body": [
        {
            "areaName": "交通管制区",
            "areaCode": "QY-1",
            "functionParams": [
                {
                    "passStrategy": "10",
                    "functionType": "TRAFFIC_CONTROL",
```

```
                "entryStrategy": "10"
            }
        ],
        "areaApexes": {
            "mapId": "1",
            "floorId": "1",
            "controlPoints": [
                {
                    "x": "14.591196172248804",
                    "y": "32.20708133971292"
                },
                {
                    "x": "24.96057416267943",
                    "y": "31.836746411483254"
                },
                {
                    "x": "24.29397129186603",
                    "y": "29.39253588516747"
                },
                {
                    "x": "14.591196172248804",
                    "y": "29.54066985645933"
                }
            ]
        },
        "locations": []
    }
]
```

```
}
```

## 6.5.2 Location Query Interface - LocationListMsg

### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be LocationListMsg. | |
| **locationCode** | string | No | Location code | **Optional. If no field is filled, the system queries all. Relations among entered conditions are "OR".** |
| **locationCodes** | List of string | No | List of location codes | |
| **locationTypes** | List of string | No | List of location types NULL_CELL (null cell), SHELF_CELL (shelf cell), E2W_PATH_CELL (cell in the east-to-west path), W2E_PATH_CELL (cell in the west-to-east path) S2N_PATH_CELL (cell in the south-to-north path), N2S_PATH_CELL (cell in | **Optional. If no field is filled, the system queries all. Relations among entered conditions are "AND".** |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | the north-to-south path), E2W_S2N_PATH_CELL (cell in the east-to-west path and south-to-north path), E2W_N2S_PATH_CELL (cell in the east-to-west path and north-to-south path) W2E_S2N_PATH_CELL (cell in the west-to-east path and south-to-north path), W2E_N2S_PATH_CELL (cell in the west-to-east path and north-to-south path), E2W_W2E_PATH_CELL (cell in the east-to-west path and west-to-east path), N2S_S2N_PATH_CELL (cell in the north-to-south path and south-to-north path), E2W_W2E_N2S_PATH_CELL (cell in the east-to-west path, west-to-east | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
| --- | --- | --- | --- | --- |
| | | | path, and north-to-south path), E2W_W2E_S2N_PATH_CELL (cell in the east-to-west path, west-to east path, and south-to-north path) N2S_S2N_E2W_PATH_CELL (cell in the north-to-south path, south-to-north path, and east-to-west path), N2S_S2N_W2E_PATH_CELL (cell in the north-to-south path, south-to-north path, and west-to-east path), OMNI_DIR_CELL (omni-directional cell), CHARGER_CELL (cell at the charging station) CHARGER_PI_CELL (cell at the charging pile power interface), BLOCKED_CELL (blocked cell), ENTRY_CELL (entrance cell), | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | EXIT_CELL (exit cell),<br>and<br>ELEVATOR_CELL<br>(elevator cell) | |
| **locationTaskStatus** | List of<br>integer | No | Location task status<br>0: idle<br>1: occupied | |
| **locationRobotStatus** | List of<br>integer | No | Location robot status:<br>0: vacant<br>1: allocated<br>2: occupied | |
| **locationContainerStatus** | List of<br>integer | No | Location container status<br>0: idle<br>1: allocated<br>2: occupied | |
| **locationFunctions** | List of<br>string | No | List of location functions<br>BACKUP_CELL (back),<br>BEEP_CELL (beep),<br>WAIT_CELL (wait),<br>AVOID_CELL (avoid),<br>SKIP_CELL (skip),<br>REST_CELL (rest),<br>TURN_CELL (turn),<br>NOT_ALLOW_TURN_C<br>ELL (not allowed to turn)<br>FIRE_PASS (firefighting),<br>RECYCLEID (recycle | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | point)<br>RESTART_CELL<br>(restart),<br>STAY_BLOCKING_CELL<br>(stay blocking) | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "11111"
    },
    "body": {
        "msgType": "LocationListMsg",
        "locationCode": "",
        "locationCodes": [],
        "locationTaskStatus": [],
        "locationRobotStatus": [],
        "locationContainerStatus": [],
        "locationTypes": [],
        "locationFunctions": []
    }
}
```


## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| | List | List of locations | |

| Field | Type | Description | Remarks |
|---|---|---|---|
| **locationCode** | string | Location code | |
| **location** | DPoint | location | Reference: 1.2 Custom Data Type |
| **index** | IPoint | Location index | Reference: 1.2 Custom Data Type |
| **hostCellCode** | string | Host location code | |
| **locationType** | string | Location type | NULL_CELL (null cell), SHELF_CELL (shelf cell), E2W_PATH_CELL (cell in the east-to-west path), W2E_PATH_CELL (cell in the west-to-east path) S2N_PATH_CELL (cell in the south-to-north path), N2S_PATH_CELL (cell in the north-to-south path), E2W_S2N_PATH_CELL (cell in the east-to-west path and south-to-north path), E2W_N2S_PATH_CELL (cell in the east-to- |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
| --- | --- | --- | --- |
| | | | west path and north-to-south path) W2E_S2N_PATH_CELL (cell in the west-to-east path and south-to-north path), W2E_N2S_PATH_CELL (cell in the west-to-east path and north-to-south path), E2W_W2E_PATH_CELL (cell in the east-to-west path and west-to-east path), N2S_S2N_PATH_CELL (cell in the north-to south path and south-to-north path), E2W_W2E_N2S_PATH_CELL (cell in the east-to-west path, west-to-east path, and north-to-south path), E2W_W2E_S2N_PATH_CELL (cell in the east-to-west path, west-to east path, and south-to-north path) |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | N2S_S2N_E2W_PATH _CELL (cell in the north-to-south path, south-to-north path, and east-to-west path), N2S_S2N_W2E_PATH _CELL (cell in the north-to-south path, south-to-north path, and west-to-east path), OMNI_DIR_CELL (omni-directional cell), CHARGER_CELL (cell at the charging pile) CHARGER_PI_CELL (cell at the charging pile power interface), BLOCKED_CELL (blocked cell), ENTRY_CELL (entrance cell), EXIT_CELL (exit cell), and ELEVATOR_CELL (elevator cell) |
| **locationTaskStatus** | integer | Location task status | 0: idle<br>1: occupied |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| **locationRobotStatus** | string | Location robot status | 0: vacant<br>1: allocated<br>2: occupied |
| **locationContainerStatus** | integer | Location container status | 0: idle<br>1: allocated<br>2: occupied |
| **length** | string | Cell length | |
| **width** | string | Cell width | |
| **allocatedRobot** | string | ID of robot allocated to the location | |
| **occupyRobot** | string | ID of robot occupying the location | |
| **allocatedContainerCode** | string | ID of container allocated to the location | |
| **occupyContainerCode** | string | ID of container occupying the location | |
| **chargerId** | string | Charging station ID | |
| **locationFunctions** | list of string | List of location functions | BACKUP_CELL (back),<br>BEEP_CELL (beep),<br>WAIT_CELL (wait),<br>AVOID_CELL (avoid),<br>SKIP_CELL (skip),<br>REST_CELL (rest),<br>TURN_CELL (turn),<br>NOT_ALLOW_TURN_ |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | CELL (not allowed to turn)<br>FIRE_PASS (firefighting),<br>RECYCLEID (recycle ID)<br>RESTART_CELL (restart),<br>STAY_BLOCKING_CELL (stay blocking) |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "msgType": "LocationListMsg",
        "code": "0",
        "msg": "Success"
    },
    "body": [
        {
            "locationCode": "11350365",
            "location": {
                "z": 1,
                "x": 124.85,
                "y": 40.15
            },
            "index": {
```

```
                "z": 1,
                "x": 113,
                "y": 36
            },
            "hostCellCode": null,
            "locationType": "W2E_PATH_CELL",
            "locationTaskStatus": "0",
            "locationRobotStatus": "0",
            "locationContainerStatus": "0",
            "length": 1.1,
            "width": 1.1,
            "direction": null,
            "dirUnload": null,
            "allocatedRobot": "",
            "occupyRobot": "",
            "allocatedContainerCode": "",
            "occupyContainerCode": "",
            "chargerId": "",
            "locationFunctions": [
                "REST_CELL"
            ]
        }
    ]
}
```

### 6.5.3 Robot Information Query Interface - RobotInfoMsg

Query robot information by conditions.

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be RobotInfoMsg. | |
| **robot** | integer | No | Robot ID | **Optional. If the field is left empty, the system will query all.** |
| **robots** | List of integer | No | List of robot IDs | |
| **robotProducts** | list of string | No | Workflow instance ID | **Relations among all query conditions are "AND".** |
| **robotConnectStatus** | integer | No | Robot connection status<br>0: all/default<br>1: normal (NORMAL, NEW_COMMER)<br>2: connection error (DISCONNECTED, CONNECTED_AGAIN) | |
| **robotTaskStatus** | integer | No | Robot task status<br>0: all/default | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | 1: operating (the robot has been assigned with at least one task that has not been removed.)<br>2: idle | |
| **robotErrorCode** | integer | No | Robot error code<br>0: all/default<br>1: normal<br>2: abnormal | |

Example of request message:


```
{
    "header": {
        "requestId": "bf672902-f0cb-498f-ac64-b668dd9a3216",
        "channelId": "11111"
    },
    "body": {
        "msgType": "RobotInfoMsg",
        "robot": "",
        "robots": [],
        "robotProducts": [
            "M1000L"
        ],
        "robotConnectStatus": "",
        "robotTaskStatus": "",
        "robotErrorCode": ""
    }
}
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| | List | Robot list | |
| **robot** | integer | Robot ID | |
| **robotProduct** | string | Robot product model | P500L (shelf lifting robot), P800L (shelf lifting robot), P800TLP (lifting robot with fixed pallet), P500R (roller robot), P800R (roller robot), M100L (SLAM navigation robot with lifting function), M100R (SLAM navigation robot with roller function), M100D (SLAM navigation robot with traction function), M100N (SLAM navigation robot without additional functions), M500L (SLAM navigation robot with lifting function), M500R (SLAM navigation robot with roller function), M800L (SLAM navigation robot with lifting function), M800R (SLAM navigation robot with roller function), |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | M1000L (SLAM navigation robot with lifting function), M1000R (SLAM navigation robot with roller function), F14L (1.4 t, 3 m-high lift forklift) F20T (2.0 t pallet truck forklift) C200S (tote-holding robot), M100PR (M100 parallel roller) |
| **robotConnectStatus** | string | Robot connection status | NEW_COMMER (new comer), DISCONNECTED (disconnected), CONNECTED_AGAIN (connected again), NORMAL (normal) SLEEPING (sleeping), REMOVED_FROM_SYSTEM (removed from system) |
| **robotStopStatus** | string | Robot stop status | NORMAL (normal), STOPPED (emergency stop), SUSPENDED (pause) |
| **robotError** | string | Robot error code | Multiple codes shall be separated with commas. |
| **powerPercent** | double | Remaining battery | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | power of the current robot | |
| **angle** | double | Robot angle | |
| **speed** | double | Real-time robot speed | |
| **location** | DPoint | location | Reference: 1.2 Custom Data Type |
| **locationIndex** | IPoint | Location index | Reference: 1.2 Custom Data Type |
| **locationCode** | string | Location code | |
| **hostLocationCode** | string | Host location code | |
| **robotPathMode** | string | Robot path model | GO_DELIVERING, // Go delivering the shelf<br>GO_RETURN, // Go returning the shelf<br>QUEUING, // Queue up<br>GO_FETCHING, // Go fetching the shelf<br>GO_CHARGING, // Go charging<br>CHARGING, // Charging<br>GO_REST, // Go to a certain location<br>FIRMWARE_UPDATE, // Update the firmware<br>IDLE, // Be idle<br>GO_RECEIVE, // Go receiving goods |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | GO_DROP, // Go dropping goods<br>GO_FETCH_BOX, // Go fetching the box<br>GO_DELIVER_BOX, // Go delivering the box<br>GO_RETURN_BOX, // Go returning the box<br>MANUAL, //Be in remote control mode<br>MAP_INIT,// Map is initialized<br>INVALID, // Be in an invalid mode<br>PALLET_GO_FETCHING, // Go fetching the pallet<br>GO_RETURN_PALLET, // Go returning the pallet<br>GO_DELIVER_PALLET; // Go delivering the pallet |
| **robotTaskId** | long | Robot task ID | |
| **path** | list of DPoint | path | Reference: 1.2 Custom Data Type |
| **voltage** | double | Battery voltage | |
| **current** | double | Battery current | |
| **temperature** | integer | Battery temperature | |
| **IP** | string | Robot IP address | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| **instanceId** | long | Workflow instance ID | |
| **taskCode** | string | Host task code/bill number | |
| **containerCode** | string | Container ID | |

Example of a response message:


```
{
    "header": {
        "responseId": "bf672902-f0cb-498f-ac64-b668dd9a3216",
        "msgType": "RobotInfoMsg",
        "code": "0",
        "requestTime": null,
        "clientCode": null,
        "msg": "调用成功"
    },
    "body": [
        {
            "robot": "1100094",
            "robotProduct": "M1000L",
            "robotConnectStatus": "NORMAL",
            "robotStopStatus": "NORMAL",
            "robotError": "",
            "powerPercent": "97",
            "angle": "-179.0",
            "speed": "1.959",
            "location": {
                "z": 4,
```

```
        "x": 21.595,
        "y": 5
    },
    "locationIndex": {
        "z": 1,
        "x": 0,
        "y": 0
    },
    "locationCode": "4019",
    "hostLocationCode": null,
    "robotPathMode": "GO_RETURN",
    "robotTaskId": 552,
    "path": [
        {
            "z": 4,
            "x": 20,
            "y": 5
        },
        {
            "z": 4,
            "x": 5,
            "y": 5
        }
    ],
    "voltage": 0,
    "current": 0,
    "temperature": 15,
    "ip": "127.0.0.1",
```

```
            "instanceId": "240",
            "taskCode": "",
            "containerCode": "00000112"
        }
    ]
}
```

### 6.5.4 Charging Station Information Query Interface - ChargerInfoMsg

Query charging station information by conditions.

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be ChargerInfoMsg. | |
| **chargerId** | integer | No | Charging station ID | **Optional. If the field is left empty, the system will query all.** |
| **chargerIds** | List of integer | No | List of charging station IDs | |

Example of request message:

```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "111111"
    },
```

```
    "body": {
        "msgType": "ChargerInfoMsg",
        "chargerId": "",
        "chargerIds": []
    }
}
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| | List | List of charging stations | |
| **chargerId** | integer | Charging station ID | |
| **hostChargerCode** | string | Host charging station ID | |
| **status** | string | Charging station status | VACANT (vacant), ALLOCATED (allocated), OCCUPIED_CHARGING (the charging station occupied by the robot is charging the robot), OCCUPIED_OFFPOWER (the charging station occupied by the robot is power off) |
| **openFlag** | integer | Charging station opening status | 1: open; 0: close |
| **location** | DPoint | Location of charging station | Reference: 1.2 Custom Data Type |
| **locationIndex** | IPoint | Location index | Reference: 1.2 Custom Data Type |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| **locationCode** | string | Location code | |
| **hostLocationCode** | string | Host location code | |
| **robot** | integer | Robot ID | -1 means that no robot is occupied. |
| **channels** | List | Charging channel list | |
| **voltage** | double | Charging voltage | |
| **current** | double | Charging current | |
| **temperature** | integer | charging temperature | |
| **status** | integer | Channel status | 0 indicates the channel is normal, and other values indicate the channel is abnormal. |
| **id** | string | Channel ID | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "msgType": "ChargerInfoMsg",
        "code": "0",
        "msg": "success"
    },
    "body": {
        "chargerId": "1",
        "hostChargerCode": "",
        "status": "VACANT",
```

```
            "openFlag": "",
            "location": "",
            "locationIndex": "",
            "locationCode": "",
            "hostLocationCode": "",
            "robot": "",
            "channels": []
        }
    }
```


## 6.5.5 Map Information Query Interface - MapInfoMsg

Query map information by conditions.

### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be MapInfoMsg. | |
| **floor** | integer | No | Floor ID | **Optional. If the field is left empty, the system will query all.** |
| **floors** | List of integer | No | List of floor IDs | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
```

```
            "channelId": "11111"
        },
        "body": {
            "msgType": "MapInfoMsg",
            "floor": "",
            "floors": []
        }
    }
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **floorSize** | integer | Number of floors | |
| **floors** | List | List of floors | |
| **floorId** | integer | Floor ID | |
| **mapType** | string | Map type | MATRIX (matrix map)<br>FUSION (fusion map) |
| **locationNumX** | integer | Number of cells in the X direction | (only displayed on the matrix map) |
| **locationNumY** | integer | Number of cells in the Y direction | (only displayed on the matrix map) |
| **locationCodes** | list | List of cells | |
| **locationCode** | string | Location code | |
| **hostLocationCode** | string | Host location code | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| **location** | DPoint | Cell location | Reference: 1.2 Custom Data Type |
| **index** | IPoint | Cell location index | Reference: 1.2 Custom Data Type |
| **locationType** | string | Cell type | NULL_CELL (null cell), SHELF_CELL (shelf cell), E2W_PATH_CELL (cell in the east-to-west path), W2E_PATH_CELL (cell in the west-to-east path) S2N_PATH_CELL (cell in the south-to-north path), N2S_PATH_CELL (cell in the north-to-south path), E2W_S2N_PATH_CELL (cell in the east-to-west path and south-to-north path), E2W_N2S_PATH_CELL (cell in the east-to-west path and north-to-south path) W2E_S2N_PATH_CELL (cell in the west-to-east path and south-to-north path), W2E_N2S_PATH_CELL (cell in the west-to-east path and north-to-south path), E2W_W2E_PATH_CELL (cell |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | in the east-to-west path and west-to-east path), N2S_S2N_PATH_CELL (cell in the north-to south path and south-to-north path), E2W_W2E_N2S_PATH_CELL (cell in the east-to-west path, west-to-east path, and north-to-south path), E2W_W2E_S2N_PATH_CELL (cell in the east-to-west path, west-to east path, and south-to-north path) N2S_S2N_E2W_PATH_CELL (cell in the north-to-south path, south-to-north path, and east-to-west path), N2S_S2N_W2E_PATH_CELL (cell in the north-to-south path, south-to-north path, and west-to-east path), OMNI_DIR_CELL (omni-directional cell), STATION_CELL (cell in the picking position of the picking station), TURN_CELL (cell in the turning position of the picking |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | station; only used in the field grid in the standard station), QUEUE_CELL (cell in the queuing position of the picking station), CHARGER_CELL (cell at the charging shelf), CHARGER_PI_CELL (cell at the charging pile power interface), BLOCKED_CELL (blocked cell), ENTRY_CELL (entrance cell), EXIT_CELL (exit cell), ELEVATOR_CELL (elevator cell), DROP_CELL (cell at the dropping point), PALLET_RACK_CELL (cell at the pallet rack), and BOX_RACK_CELL (cell at the box rack) |
| **locationRobotStatus** | string | Cell status | VACANT (vacant) ORDERED (ordered) ALLOCATED (allocated) OCCUPIED (occupied) OCCUPIED_BY_SHELF |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | (occupied by shelf) |
| **length** | float | Cell length | |
| **width** | float | Cell width | |
| **floorId** | integer | Floor ID | |
| **allocatedRobot** | integer | ID of robot assigned to the cell | |
| **occupyRobot** | integer | ID of robot occupying the cell | |
| **occupiedContainerCode** | string | ID of container occupying the cell | |
| **chargerId** | integer | Charging station ID | |
| **locationFunctions** | list of string | List of cell functions | BACKUP_CELL (back), BEEP_CELL (beep), WAIT_CELL (wait), AVOID_CELL (avoid), SKIP_CELL (skip), REST_CELL (rest), TURN_CELL (turn), NOT_ALLOW_TURN_CELL (not allowed to turn), FIRE_PASS (firefighting), RECYCLEID (recycle ID), RESTART_CELL (restart), |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | STAY_BLOCKING_CELL<br>(stay blocking) |

Example of a response message:


```
{
    "header": {
        "responseId": "662303f7-1b1b-499a-86b6-5140b61fcf25",
        "msgType": "MapInfoMsg",
        "code": "0",
        "requestTime": null,
        "clientCode": null,
        "msg": "调用成功"
    },
    "body": [
        {
            "floorSize": 5,
            "floors": [
                {
                    "floorId": 1,
                    "mapType": "FUSION",
                    "locationNumX": 0,
                    "locationNumY": 0,
                    "locationCodes": [
                        {
                            "floorId": 1,
                            "length": 0.8,
                            "locationCode": "116",
                            "locationType": "CHARGER_CELL",
```

```
            "locationRobotStatus": "VACANT",
            "width": 0.8,
            "hostLocationCode": null,
            "occupiedContainerCode": null,
            "occupyRobot": null,
            "locationFunctions": null,
            "allocatedRobot": null,
            "chargerId": 3,
            "index": null,
            "location": {
                "z": 1,
                "x": 46.121,
                "y": 32.893
            }
        },
        {
            "floorId": 1,
            "length": 0.8,
            "locationCode": "118",
            "locationType": "STATION_CELL",
            "locationRobotStatus": "VACANT",
            "width": 0.8,
            "hostLocationCode": null,
            "occupiedContainerCode": null,
            "occupyRobot": null,
            "locationFunctions": [
                "TURN_CELL"
            ],
```

```
                    "allocatedRobot": null,
                    "chargerId": null,
                    "index": null,
                    "location": {
                        "z": 1,
                        "x": 101.46,
                        "y": 24.334
                    }
                }
            ]
        }
    ]
    }
  ]
}
```


### 6.5.6 Fire Emergency Stop Interface - RobotControlMsg

When fire emergency stop is triggered, the robot stops moving but components on the robot body can continue actions. In this mode, the robot will keep away from the fire passage.

This interface is a synchronous response interface. Successful return means that the user has received the request from the host to set robot management to emergency stop.

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be RobotControlMsg. | |
| **instruction** | string | Yes | Implementation instruction: The value must be FIRE_STOP. | |
| **areaCodes** | list of string | No | List of area codes | If this field is empty, global fire emergency stop is enabled. |

Example of request message:

```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01"
    },
    "body": {
        "msgType": "RobotControlMsg",
        "instruction": "FIRE_STOP",
        "areaCodes": [
            "qy01",
            "qy02"
        ]
    }
}
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **success** | list | List of successes | |
| **areaCode** | string | Area code of success. If global fire stop is enabled, this field is empty. | |
| **result** | string | Result. In the case of success, this field is empty. | |
| | | | |
| **fail** | list | List of failures | |
| **areaCode** | string | Area code of failure. If global fire stop is enabled, this field is empty. | |
| **result** | string | Reason for exception of passed area | No areaCode - The area does not exist.<br>No STOP function - There is no emergency stop function in the area. |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "msgType": "RobotControlMsg",
        "code": "5001",
        "msg": "区域编码全部或者部分不存在"
    },
    "body": {
        "success": [],
```

```
            "fail": [
                {
                    "areaCode": "qy01",
                    "result": "No areaCode"
                },
                {
                    "areaCode": "qy02",
                    "result": ""
                }
            ]
        }
    }
```

## 6.6 Traffic Control Area Interfaces

### 6.6.1 Traffic Control Area Query Interface - TrafficControlAreaListMsg

The host system acquires the vehicle and queuing information in the traffic control area from GMS through this interface.

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be TrafficControlAreaListMsg. | |
| **areaName** | string | No | Area name | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **areaCode** | string | No | Area code | **Optional. If the field is left empty, the system will query all. Enter the conditions. In the case of existence, consistent information with the filled conditions is returned. Otherwise, nothing is returned.** |
| **areaNames** | list of string | No | List of area names | |
| **areaCodes** | list of string | No | List of area codes | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId": "client-01",
        "clientCode": "geekplus"
    },
    "body": {
        "msgType": "TrafficControlAreaListMsg",
        "areaName": "",
        "areaCode": "",
        "areaNames": [
```

```
                1,
                2,
                3
            ],
            "areaCodes": [
                "a",
                "b",
                "c"
            ]
        }
    }
```

## Response message

The fields of the response message body are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **areaName** | string | Area name | |
| **areaCode** | string | Area code | |
| **areaStatus** | string | Area status | Lock: lock<br>Release: release<br>PhysicalLock: lock triggered by host system |
| **containRobs** | list | List of contained robots | |
| **robotId** | string | Robot ID | |
| **clientCode** | string | Source/manufacturer | geekplus protocol is adopted by Geek+<br>Protocols of other manufacturers |
| **robStatus** | string | Robot status | Wating: waiting for entry<br>Pause: suspend entry |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | | Entrying: entering<br>MovingInArea: moving in the area |
| **applyTime** | string | Application time for entering | Format: yyyy-MM-dd HH:mm:ss |
| **inTime** | string | Entering time | Format: yyyy-MM-dd HH:mm:ss |
| **entryPoint** | string | Entry location | |

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee3",
        "clientCode": "geekplus",
        "msgType": "TrafficControlAreaListMsg",
        "code": "0",
        "msg": "success"
    },
    "body": [
        {
            "areaName": "1",
            "areaCode": "1",
            "areaStatus": "Lock",
            "containRobs": [
                {
                    "robotId": "20",
                    "clientCode": "geekplus",
                    "robStatus": "MovingInArea",
                    "inTime": "2020-10-28    00:00:00",
```

```
                "entryPoint ": "east"
            },
            {
                "robotId": "21",
                "clientCode": "geekplus",
                "robStatus": "Wating",
                "applyTime": "2020-10-28    00:01:00",
                "entryPoint": "east"
            }
        ]
    }
  ]
}
```


### 6.6.2 Interface of Applying for Entering/Leaving Traffic Control Area (Synchronous Interface) - CrossAreaPermissionMsg

GMS makes synchronous response and asynchronous response to the request of applying for entering/leaving the traffic control area.

This section describes synchronous responses, while the next section describes asynchronous responses.

Either response method can be selected during project implementation, but the asynchronous interface must be selected if interaction with external facilities is involved when entering/leaving the traffic control area, for example when opening or closing the grating door.

Only the synchronous interface or asynchronous interface can be selected in the same project, not both.

This document shall not be disseminated in any form without the written permission of Geek+.

The applicant shall send such information as the manufacturer/source, area code, robot ID, and taskCode to GMS through this interface before the robot enters/leaves the traffic control area, while GMS shall return the permission. Through this method, the host system and GMS respond to each other synchronously. However, when GMS gives a feedback that the robot is not permitted to enter/leave the traffic control area or connection failure occurs, the applicant shall initiate the request for polling.

## Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be CrossAreaPermissionMsg. | |
| **instruction** | string | Yes | execution instruction | ENTRY: entry<br>LEAVE: leave |
| **areaCode** | string | Yes | Area code | |
| **robotId** | string | No | Robot ID | If this field is empty, the entering sequence is not controlled, and feedback is given in the application time sequence. |
| **entryPoint** | string | No | Entry point | |
| **priority** | integer | No | Priority of task: The smaller the value, the higher the priority. 1 represents the | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | highest priority. | |
| **taskCode** | string | No | Task ID | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId ": "11111",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "CrossAreaPermissionMsg",
        "instruction": "Entry",
        "areaCode": "A",
        "robotId": "06",
        "entryPoint": "",
        "priority": "1",
        "taskCode": ""
    }
}
```


## Response message

The fields of the response message header are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **code** | string | Response code:<br>0 - permitted<br>100 - Not permitted to enter the locked area; | |

| Field | Type | Description | Remarks |
|---|---|---|---|
| | | 101 - Failed because the area is not found;<br>102 - Failed. Please try again later.<br>For details of errors, see the code forms. | |
| **msg** | string | Response message description:<br>Permitted;<br>Not permitted to enter the locked area;<br>Failed because the area is not found;<br>Failed. Please try again later. | |

The response message body is empty.

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ads256",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "CrossAreaPermissionMsg",
        "code": "0",
        "msg": "允许"
    }
}
```


### 6.6.3 Interface of Applying for Entering/Leaving Traffic Control Area (Asynchronous Interface) - CrossAreaPermissionBMsg

The applicant shall send such information as the manufacturer/source, area

code, robot ID, and taskCode to GMS through this interface before the robot enters/leaves the traffic control area, while GMS shall send the synchronous response message, which only indicates that it has received the request from the host system. GMS will give a feedback on permission by calling the asynchronous callback interface - CrossAreaApproveMsg.

## Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be CrossAreaPermissionBMsg. | |
| **instruction** | string | Yes | execution instruction | ENTRY: entry<br>LEAVE: leave |
| **areaCode** | string | Yes | Area code | |
| **robotId** | string | No | Robot ID | If this field is empty, the entering sequence is not controlled, and feedback is given in the application time sequence. |
| **entryPoint** | string | No | Entry point | |
| **priority** | integer | No | Priority of task: The smaller the value, the higher the priority. 1 represents the highest priority. | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **taskCode** | string | No | Task ID | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId ": "11111",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "CrossAreaPermissionBMsg",
        "instruction": "Entry",
        "areaCode": "A",
        "robotId": "06",
        "entryPoint": "",
        "priority": "1",
        "taskCode": ""
    }
}
```


## Response message

The response message body is empty.

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ads256",
```

```
            "clientCode": "geekplus",
            "requestTime": "2020-06-06 18:03:24",
            "msgType": "CrossAreaPermissionBMsg",
            "code": "0",
            "msg": "Success"
        }
    }
```


### 6.6.4 Interface of Canceling Application for Entering/Leaving Traffic Control Area - CrossAreaCancelMsg

For the request of entering/leaving the area notified by GMS through asynchronous interface, the applicant shall call this interface for feedback when robot entering/leaving is canceled.

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be CrossAreaCancelMsg. | |
| **instruction** | string | Yes | execution instruction | ENTRY: entry<br>LEAVE: leave |
| **areaCode** | string | Yes | Area code | |
| **robotId** | string | No | Robot ID | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **taskCode** | string | No | Task ID | |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId ": "11111",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "CrossAreaCancelMsg",
        "instruction": "Entry",
        "areaCode": "A",
        "robotId": "06",
        "taskCode": ""
    }
}
```


## Response message

The fields of the response message header are as follows:

| Field | Type | Description | Remarks |
|---|---|---|---|
| **code** | string | Response code:<br>0: succeeded<br>100: cancellation failed<br>For details of errors, see the code forms. | |
| **msg** | string | Response message description: | |

The response message body is empty.

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ads256",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "CrossAreaCancelMsg",
        "code": "0",
        "msg": "Success"
    }
}
```


### 6.6.5 Interface of Approval of Entering/Leaving Traffic Control Area through Asynchronous Interface - CrossAreaApproveMsg

GMS notifies the host system whether the robot is permitted to enter/leave the area through the callback interface. After receiving such a request, in addition to giving a feedback on receiving through synchronous response, the host system shall give a feedback on robot entering/leaving after the robot enters the area through the asynchronous interface AreaMovingPhaseMsg.

**Note: This interface is a callback interface for GMS to call the host system. This interface shall be developed by the host system in corresponding format.**

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | - | Message type: The value must be CrossAreaApproveMsg. | |
| **instruction** | string | - | execution instruction | ENTRY: entry<br>LEAVE: leave |
| **areaCode** | string | - | Area code | |
| **robotId** | string | - | Robot ID | |
| **entryPoint** | string | - | Entry point | |
| **taskCode** | string | - | Task ID | |
| **code** | string | - | Application result | 0 - permitted<br>100 - Not permitted to enter the locked area;<br>101 - Failed because the area is not found;<br>102 - Failed. Please try again later.<br>For details of errors, see the code forms. |

Example of request message:


```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee5",
        "entryrequestId": "9194e002ad324176974ae9438a011ee3",
        "channelId ": "11111",
        "clientCode": "geekplus",
```

```
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "CrossAreaApproveMsg",
        "instruction": "Entry",
        "areaCode": "10001",
        "robotId": "06",
        "entryPoint": "",
        "taskCode": "",
        "code": "0"
    }
}
```

## Response message

**Note: The host system shall respond in corresponding format.**

The response message body is empty.

Example of a response message:

```
{
    "header": {
        "responseId": "9194e002ad324176974ae9438a011ee5",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "CrossAreaApproveMsg",
        "code": "0",
        "msg": "Success "
    }
}
```

This document shall not be disseminated in any form without the written permission of Geek+.

### 6.6.6 Passing Status Feedback Interface - AreaMovingPhaseMsg

The moving state of the traffic control area contains two types: entered and left.

Two judgment methods for robot leaving are available in GMS:

1. Automatic release: The designated manufacturer/source automatically releases the robot at the agreed time to save the robot from submitting the leaving feedback, and the system deems that the robot has left the traffic control area after the set countdown.
2. Automatic leaving feedback: After leaving the traffic control area (which must be confirmed), the robot will give a feedback on leaving by sending the manufacturer/source, area ID, and robot ID to GMS through this interface. This interface adopts the second method.

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be AreaMovingPhaseMsg. | |
| **instruction** | string | Yes | execution instruction | ENTRY: entered<br>LEAVE: left |
| **areaCode** | string | Yes | Area code | |
| **robotId** | string | No | Robot ID | |
| **entryPoint** | string | No | Entry point | |
| **taskCode** | string | No | Task ID | |

Example of request message:

```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
        "channelId ": "11111",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "AreaMovingPhaseMsg",
        "instruction": "ENTRY",
        "areaCode": "A",
        "robotId": "06",
        "entryPoint": "",
        "priority": "1",
        "taskCode": ""
    }
}
```


## Response message

The response message body is empty.

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ads256",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "AreaMovingPhaseMsg",
        "code": "0",
```

```
            "msg": "Success"
        }
    }
```

### 6.6.7 Traffic Control Area Locking/Releasing Interface - TrafficAreaLockMsg

If the traffic control area shall be locked due to exception and other external factors, the user shall call this interface instruction and select LOCK to lock the area. The locked area turns to physical lock status to stop the robot from entering and make all applications in a queue. After calling this interface instruction again and selecting UNLOCK, the area turns to the normal entering status.

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | Yes | Message type: The value must be TrafficAreaLockMsg. | |
| **instruction** | string | Yes | execution instruction | LOCK: lock<br>UNLOCK: unlock |
| **areaCode** | string | Yes | Area code | |

Example of request message:

```
{
    "header": {
        "requestId": "9194e002ad324176974ae9438a011ee3",
```

```
        "channelId ": "11111",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24"
    },
    "body": {
        "msgType": "TrafficAreaLockMsg",
        "instruction": "LOCK",
        "areaCode": "A"
    }
}
```

## Response message

The response message body is empty.

Example of a response message:

```
{
    "header": {
        "responseId": "9194e002ad324176974ads256",
        "clientCode": "geekplus",
        "requestTime": "2020-06-06 18:03:24",
        "msgType": "TrafficAreaLockMsg",
        "code": "0",
        "msg": "Success"
    }
}
```

This document shall not be disseminated in any form without the written permission of Geek+.

## 6.7 Callback Interface

### 6.7.1 Task Callback Interface - MovingCallbackMsg

1. Besides tasks originated through the request interface, the tasks originated through the operation interface of the workstation end can also be called back.
2. The callback task phase is divided into three levels, namely workflowPhase (first level), taskPhase (second level), and robotPhase (third level).
3. Note: During task implementation, the "interrupt wait" and "implement instruction" in taskPhase are changes in the task phase that are not triggered by the robot. Therefore, at this time, the robotPhase is empty. If there is a list of events to be implemented after "interrupt wait", the task phase is changed to "implement instruction". At this time, the callback of robotPhase is created according to the change in the robotPhase.

**Note: This interface is a callback interface for GMS to call the host system. This interface shall be developed by the host system in corresponding format.**

#### Request message

The fields of the request message body are as follows:

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **msgType** | string | - | Message type: The value must be MovingCallbackMsg. | |
| **workflowCode** | string | - | Workflow code | |
| **taskCode** | string | - | Host task code/bill number | |
| **instanceId** | long | - | Workflow instance ID | |
| **parentInstanceId** | long | - | Parent instance ID | |

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| **instancePriority** | integer | - | Priority of workflow instance (task): The smaller the value, the higher the priority. 1 represents the highest priority. | |
| **taskId** | long | - | Subtask ID | |
| **robotTaskId** | long | - | Robot task ID | |
| **containerCode** | string | - | Container ID | |
| **scanCode** | string | - | Pallet code | If the point-to-point pallet code field failed to be scanned and verified, the code will be updated to the scanned code value.<br>If this field is not scanned, the code value is empty. |
| **scanningInformation** | string | - | Reason for scanning error | Code comparison failed, and no code is scanned. |
| **robot** | integer | - | Robot ID | |
| **nodeCodeFrom** | string | - | Node code configured in the workflow at the start point of task | |
| **nodeCodeTo** | string | - | Node code configured in the workflow at the end point of task | |
| **locationFrom** | string | - | Actual start location code | It represents the map node code. If the host code exists, |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | return such code; otherwise return the GMS code. |
| **locationTo** | string | - | Actual destination location code | It represents the map node code. If the host code exists, return such code; otherwise return the GMS code. |
| **waitLocation** | string | - | Wait point code | |
| **waitDir** | string | - | Robot orientation at wait point | |
| **waitNextLocation** | string | - | Next location of wait point | |
| **workflowPhase** | string | - | Workflow status | CREATE: create 10<br>EXECUTION: being implemented 20<br>COMPLETED: completed 30<br>CLOSED: exception/closed 31<br>CANCELED: canceled 32 |
| **taskPhase** | string | - | Task phase | CREATE: create 10<br>WAIT_QUEUE_ROBOT: waiting for queuing 13<br>WAITING_SENT: waiting for sending 15<br>ISSUED: issued 16<br>EXECUTION: being implemented 20<br>WAITING_NODE: waiting at |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | node 25<br>PAUSE: pause 26<br>CANCLE: cancel 27<br>LAST_STEP: last step 28<br>CANCELEXECUTING: cancellation in progress 29<br>COMPLETED: completed 30<br>CLOSED: exception/closed 31<br>CANCELED: canceled 32<br>COMMAND_EXECUTING: task implementation in progress 22<br>PASS_WAIT_POINT: confirming to leave 23<br>TASK_UNIT_COMPLETE: unit task completed 36<br>CANCEL_FAILED: cancellation failed 34<br>BREAK_FAILED: unit task interruption failed 35 |
| **robotPhase** | string | - | Robot task phase | GO_FETCHING: The robot is fetching the shelf (container).<br>SHELF_FETCHED: The robot has fetched the shelf.<br>GO_DELIVERING: The robot is moving the shelf. |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | QUEUING: The robot is queuing up.<br>SHELF_ARRIVED: The shelf has arrived at the destination.<br>GO_RETURN: The robot is returning the shelf.<br>SHELF_TURNING: The shelf is rotating.<br>MOVING: The robot is moving.<br>ARRIVED_WAIT_POINT: The robot has arrived at the wait point.<br>LEAVING_WAIT_POINT: The robot has left the wait point.<br>RECEIVE_FINISH: The roller robot has received the goods.<br>DROP_FINISH: The roller robot has dropped the goods.<br>ARRIVED_ELEVATOR_ENTRY: The robot has arrived at the elevator entrance, waiting for entering the elevator. |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | ENTERED_ELEVATOR: The robot has stepped into the elevator.<br>LEAVED_ELEVATOR: The robot has left the elevator.<br>ARRIVED: The robot has arrived at the destination.<br>FETCHED: The task has been completed.<br>ARRIVED_AREA_ENTRY: The robot requests entry into the area.<br>ENTERED_ AREA: The robot has entered the area.<br>LEAVED_ AREA: The robot has left the area.<br>TURN_FINISH: Shelf rotation is finished.<br>TO_WAITBIT: Go to waiting position<br>ARRIVED_WAITBIT: The robot has arrived at the waiting location.<br>ARRIVED_OPERATE: The robot arrives at the docking location.<br>OPERATE_ENTRY: The robot requests to enter the docking |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | | location.<br>OPERATE_ALLOWED: The robot is ready to enter the docking location.<br>ADJUST_OPERATE: The robot has adjusted its posture at the docking location.<br>OPERATE_FINISHED: Docking /action is finished at the docking location (machine).<br>OPERATE_UNIT_COMPLETED: The robot leaves the docking location after the unit action is finished.<br>GO_OFFSET_FINISH: Front and rear offset of the co-robots is finished. |
| **robotError** | string | - | Robot task error | For details, see: 8.3 Robot Exception Codes |
| **workflowExceptionCode** | string | - | Workflow exception code | |
| **workflowExceptionDesc** | string | - | Workflow exception description | |
| **extraParams** | object | - | There are up to 20 extra parameters. A parameter or multiple parameters can be passed. Extra parameters are issued through upstream tasks, and returned through the callback message. Extra parameters | |

This document shall not be disseminated in any form without the written permission of Geek+.

| Field | Type | Required | Description | Remarks |
|---|---|---|---|---|
| | | | issued through upstream tasks are displayed, but non-issued extra parameters and the extra parameters not used by ARK are not displayed. If there are no extra parameters, extraParams are not displayed (as the case may be). | |
| **extraParam1** | string | - | | |
| **extraParam2** | string | - | | |
| **extraParam3** | string | - | | |
| **extraParam4** | string | - | | |
| **extraParam5** | string | - | | |
| **extraParam6** | string | - | | |
| **extraParam7** | string | - | | |
| **extraParam8** | string | - | | |
| **...** | | | | |
| **extraParam19** | string | - | | |
| **extraParam20** | string | - | | |

Example of request message:


```
{
    "header": {
        "clientCode": "geekplus",
        "requestId": "07681352-aabc-4921-862f-d635185fa8a4",
        "userKey": "GeekPlus",
        "version": "1.0.0",
        "warehouseCode": "GeekPlus"
    },
```

```
    "body": {
        "containerCode": "00000112",
        "instanceId": 241,
        "instancePriority": 1,
        "locationFrom": "-1",
        "locationTo": "4020",
        "msgType": "MovingCallbackMsg",
        "nodeCodeFrom": "-1",
        "nodeCodeTo": "4020",
        "robot": "1100094",
        "robotPhase": "GO_FETCHING",
        "robotTaskId": 554,
        "scanValidate": true,
        "taskCode": "",
        "taskId": 525,
        "taskPhase": "20",
        "workflowCode": "F000064",
        "workflowPhase": 20
    }
}
```


## Response message

**Note: The host system shall respond in corresponding format.**

The response message body is empty.

Example of a response message:


```
{
    "header": {
        "responseId": "9194e002ad324176974ads256",
```

```
            "clientCode": "geekplus",
            "requestTime": "2020-06-06 18:03:24",
            "msgType": "MovingCallbackMsg",
            "code": "0",
            "msg": "success"
        },
        "body": {}
    }
```


# 7 Usage Rules

## 7.1 Area

For the point-to-point moving interface, interface of moving from a point to multiple points, and workflow start interface, when the start point is a **region**, the types of containers that can be stored in the area shall be maintained in the system first.

1. If the container ID is designated, the container type will not be verified.
2. If the container ID and the container type are designated at the same time, the container ID shall prevail, and the container type will be ignored.
3. If a type of container is configured in the area, it is unnecessary for the interface to pass this container type.
4. If there is a type of container in the area, the existing container type will be selected by default, without the need to pass the container type.
5. If multiple types of containers are configured in the area:
    a. The designated container type shall be passed to the interface.
    b. If the container type is not passed to the interface and there is no container or there are multiple types of containers in the area, an

error prompt message will be returned.

c. If the container type is not passed to the interface and there is only one type of container in the area, this type of container will be automatically allocated.

# 8 Reason Codes

## 8.1 Reason Codes

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 1 | General | 0 | Calling succeeded |
| 2 | | -100 | Unknown meaning |
| 3 | | 100 | Failure due to unknown reason and exception |
| 4 | | 1000 | The header is empty. |
| 5 | | 1001 | Message type error |
| 6 | | 1002 | Message format error |
| 7 | | 1003 | Repeated requestId. |
| 8 | | 1004 | The channelId cannot be empty. |
| 9 | | 1005 | The requestid cannot be empty. |
| 10 | | 1006 | The requestid already exists. |
| 11 | | 1007 | The msgType of the request message cannot be empty. |
| 12 | | 1008 | The msgType of the request message is not supported. |
| 13 | | 1100 | The server where the current request is made is not the main server! |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 14 | | 1200 | The implementation instruction cannot be empty. |
| 15 | | 1201 | Unsupported implementation instruction type |
| 16 | Moving task | 1205 | The start location code cannot be empty. |
| 17 | | 1206 | Failed to match the location by any random point, workstation, and area in the system according to designated start location code. |
| 18 | | 1210 | The target location code cannot be empty. |
| 19 | | 1211 | The target location code does not exist. |
| 20 | | 1212 | Failed to match the location by any random point, workstation, and area in the system according to designated target location code. |
| 21 | | 1215 | The set of target locations cannot be empty. |
| 22 | | 1220 | The flow strategy cannot be empty. |
| 23 | | 1221 | Unsupported flow strategy |
| 24 | | 1230 | Emergency stop failed. |
| 25 | | 1231 | System recovery failed. |
| 26 | | 1232 | Robot emergency stop failed. |
| 27 | | 1233 | Robot recovery failed. |
| 28 | | 1234 | Format error of passed needTime |
| 29 | | 6000 | General interactive node configuration does not exist. |
| 30 | | 6005 | Interactive node configuration does not exist. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 31 | | 6010 | Successful processing logic of interactive node configuration is empty. |
| 32 | | 6011 | Successful processing logic of interactive node configuration is automatic triggering. |
| 33 | | 6020 | Component instruction of interactive node configuration is empty. |
| 34 | | 6021 | Component instruction of interactive node configuration is not supported. |
| 35 | | 6022 | The robot failed to implement the component instruction of interactive node configuration. |
| 36 | | 6031 | The instruction implementation phase is invalid. |
| 37 | | 6040 | Instruction implementation failed. |
| 38 | | 9500 | The template does not exist. |
| 39 | Workflow implementation | 4000 | The point in the area is empty. |
| 40 | | 4001 | The point in the area does not exist. |
| 41 | | 4002 | There is no idle point in the area. |
| 42 | | 5001 | The workflow priority value must be an integer above 0. |
| 43 | | 5300 | There is no matching workflow. |
| 44 | | 5302 | The matching workflow is not unique. |
| 45 | | 5400 | No workflow is matched according to the start point. |
| 46 | | 5401 | No node code is found in the system according to the start point code. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 47 | | 5402 | The first node of the workflow is not acquired. |
| 48 | | 5403 | The first node does not match the area where the container is located. |
| 49 | | 5404 | The first node does not match the cellCode where the container is located. |
| 50 | | 5405 | The first node does not match the workstation where the container is located. |
| 51 | | 5406 | The container type at the first node does not match that started in the workflow. |
| 52 | | 5407 | The first node is virtual, and the container ID cannot be empty. |
| 53 | | 5408 | Either the start location code or container ID cannot be empty. |
| 54 | | 5410 | The flow parameter is invalid. |
| 55 | | 5500 | The configured workflow node does not exist. |
| 56 | | 5501 | The workflow cannot be started because there are fewer than two sets of workflow nodes. |
| 57 | | 5505 | No node code is found in the system according to the demand point code. |
| 58 | | 5506 | Failed to match the workflow according to the demand point. |
| 59 | | 5510 | The set of next nodes of the first node is empty. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 60 | | 5511 | The next node of the first node is not unique. |
| 61 | | 5512 | The next node of the first node does not match the node in the system. |
| 62 | | 5513 | The designated next node does not exist in the system. |
| 63 | | 5514 | The node interaction is manually selected, but the next node is empty. |
| 64 | | 5515 | The node interaction is not manually selected, but the flow rule is empty. |
| 65 | | 8000 | The instance ID, task ID, or subtask ID must be designated. |
| 66 | | 8001 | The workflowWorkId cannot be empty. |
| 67 | | 8005 | The task ID is empty. |
| 68 | | 8006 | The task ID already exists. |
| 69 | | 8007 | The instance corresponding to the task ID does not exist. |
| 70 | | 8010 | The task instance does not exist. |
| 71 | | 8011 | Failed to proceed with the operation because the task instance has already been completed. |
| 72 | | 8012 | Multiple tasks exist. |
| 73 | | 8020 | The task status cannot be empty. |
| 74 | | 8021 | The task status is invalid! |
| 75 | | 8022 | Unable to proceed to the next step under the current task status! |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 76 | | 8030 | The priority adjustment mode cannot be empty. |
| 77 | | 8031 | The priority adjustment mode is not supported. |
| 78 | | 8041 | The task corresponding to the task ID does not exist. |
| 79 | | 8051 | Exception in corner turning and side turning parameters |
| 80 | | 8052 | Angle turning and side turning Exception in side parameters |
| 81 | | 8053 | Angle turning and side turning Exception in angle parameters |
| 82 | | 8054 | Current task status exception during angle turning |
| 83 | | 8055 | The presequence step is missing. |
| 84 | | 8056 | The next code is undefined. Please set the specific node first! |
| 85 | | 9000 | The default rule does not exist. |
| 86 | | 9006 | The robot task does not exist. |
| 87 | | 9007 | Failed to match the unique robot task. |
| 88 | | 9020 | The task being implemented at the wait point does not exist. |
| 89 | | 9022 | Failed to proceed to the next step of the task at the wait point. |
| 90 | Container removal and | 4600 | Failed to match the container type. |
| 91 | | 4601 | The container type must be designated. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 92 | addition | 4602 | Container type does not exist |
| 93 | | 4603 | The container type in the system is not unique and must be designated. |
| 94 | | 4604 | Failed to match the unique container type. |
| 95 | | 4605 | Container type does not match. |
| 96 | | 4631 | Container ID cannot be empty. |
| 97 | | 4632 | The container does not exist. |
| 98 | | 4633 | The container on the cell does not exist. |
| 99 | | 4634 | The container ID must be 1-16 characters long and can only contain letters and digits! |
| 100 | | 4650 | The number of containers added in batch is 1 by default and 5000 at most. |
| 101 | | 4800 | Both the inbound location and container ID are empty. |
| 102 | | 4801 | The container already exists at the destination location. |
| 103 | | 4802 | Failed to generate the container ID. |
| 104 | | 4804 | Failed to add the container. |
| 105 | | 4805 | The container already exists. |
| 106 | | 4910 | The container has already been removed. |
| 107 | | 4911 | There is an ongoing task on the container. |
| 108 | | 4912 | Failed to remove the container from the map. |
| 109 | | 4914 | Empty status and full status must be passed. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 110 | | 4915 | The list of container IDs and the number of containers generated cannot be empty at the same time. |
| 111 | | 4916 | The last digit (or digits) of the start code of new container shall be number(s). |
| 112 | | 4917 | The number of containers generated must be more than 0. |
| 113 | | 4918 | The initial shelf angle ranges from -180.0° to 180.0°. |
| 114 | | 4921 | There are containers in use at the destination location or the destination location is being occupied by a task. |
| 115 | Workstation queuing | 3500 | Workstation does not exist. |
| 116 | | 3501 | The workstation code cannot be empty. |
| 117 | | 3507 | The rest point in the workstation is not unique and must be designated. |
| 118 | | 3510 | Workstation queuing control does not exist. |
| 119 | | 3511 | Workstation queuing control is enabled and repeated enabling is not allowed. |
| 120 | | 3512 | Workstation queuing control is disabled and repeated disabling is not allowed. |
| 121 | | 3513 | Workstation queuing control status is undefined. |
| 122 | | 3515 | The information of the rest point in the workstation is empty. |
| 123 | | 3516 | The rest point in the workstation does not |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| | | | match. |
| 124 | Traffic control | -2 | Enter the area code. |
| 125 | | -3 | Enter the manufacturer information. |
| 126 | | 100 | Request rejected because the area is locked. |
| 127 | | 101 | Entry failed because the area is not found. |
| 128 | | 102 | Entry failed. Please try again later. |
| 129 | | 103 | The Robot ID must be added to the synchronous interface. |
| 130 | | 104 | Failed to receive the task. |
| 131 | | 105 | The entry point must be added to the synchronous interface. |
| 132 | | 106 | The task ID cannot be empty. |
| 133 | | 107 | The area ID cannot be empty. |
| 134 | | 109 | The current task status is incorrect. |
| 135 | | 120 | Task does not exist |
| 136 | | 200 | Failed to update the "left" status because of no leaving task in the area. |
| 137 | | 201 | This system is subject to asynchronous traffic control. Please use the asynchronous interface. |
| 138 | | 202 | No traffic control information is configured in the area. |
| 139 | | 203 | Failed to update the "entered" status because of no entering task in the area. |
| 140 | | 204 | Failed to update the "left" status because of |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| | | | no running task in the area. |
| 141 | | 205 | This system is subject to synchronous traffic control. Please use the synchronous interface. |
| 142 | | 206 | This task already exists. Please do not request for the same task again. |
| 143 | | 207 | Configure the manufacturer callback address: Approve entry. |
| 144 | | 208 | No task available whose entry can be canceled |
| 145 | | 209 | Entry cancellation failed because the robot under this task has already entered the area. |
| 146 | | 210 | No task ready for entry exists |
| 147 | | 211 | The current strategy applies to a single entrance. Please supplement the entrance information. |
| 148 | | 212 | There are multiple current tasks. |
| 149 | | 213 | Request rejected because the current task is not in waiting for entry phase |
| 150 | | 214 | No task available for cancellation |
| 151 | | 215 | The current request task does not exist. |
| 152 | | 216 | Failed to apply for leaving because the current task is not within the traffic control area. |
| 153 | | 217 | Failed to apply for leaving. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 154 | | 218 | Application for leaving has already been made. Please do not submit a repeated application. |
| 155 | | 219 | Enter the area ID. |
| 156 | | 300 | The robot must be in "operation in managed area" status. |

## 8.2 RMS Management Exception Codes

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 1 | System task fault code | 20007 | Path planning failed. |
| 2 | | 20009 | The robot is not on the map. |
| 3 | | 20004 | The start point or end point of the task is an obstacle. |
| 4 | | 20006 | There are robots or shelves in the path which are obstacles. |
| 5 | | 20008 | The path resources are likely being occupied. |
| 6 | | 20005 | No path can be planned. |
| 7 | | 20002 | Robot connection failed. |
| 8 | | 20000 | Subtask sending timeout |
| 9 | | 20001 | Instruction sending timeout |
| 10 | | 20010 | Subtask exception |
| 11 | | 20011 | Timeout of shelf fetching by robot |
| 12 | | 20012 | Fetch the shelf before reaching the shelf location. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 13 | | 20013 | Failed to arrive at the wait point after a long time. |
| 14 | | 20014 | The battery level of the robot does not increase after a long time. |
| 15 | | 20015 | The robot reaches the low electric ratio. |
| 16 | | 20016 | Failed to implement the charging task because the charging station does not match. |
| 17 | | 20017 | Failed to implement the charging task because of no idle charging station. |
| 18 | | 20018 | Failed to implement the charging task because of unavailable charging station. |
| 19 | | 20019 | The charging station cell is occupied by other robots. |
| 20 | | 20020 | Failed to charge the robot across the area. |
| 21 | | 20021 | Failed to charge the robot across the floor. |
| 22 | | 20022 | Failed to implement the charging task because the current task is blocked. |
| 23 | | 20023 | Failed to implement the charging task because the current task is not finished. |
| 24 | | 20024 | The charging time is too long. |
| 25 | | 20025 | Timeout for sending the charging station start instruction |
| 26 | | 20026 | Timeout for sending the charging station ending instruction |
| 27 | | 20027 | The battery level of robot is too high. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 28 | | 20028 | Failed to find the rest location. |

## 8.3 Robot Exception Codes

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 1 | 0102 frame type | | |
| 2 | Communication fault code | 100001 | The communication link between the obstacle detection box and main control encountered error. |
| 3 | | 100002 | The obstacle detection box/host computer received incorrect data from the obstacle detector. |
| 4 | | 100003 | The communication link between the power module and main control encountered error. |
| 5 | | 100004 | The power module received incorrect battery data. |
| 6 | | 100005 | CAN1 communication error. For example, no data is received within 400 ms. |
| 7 | | 100006 | CAN2 communication error. For example, no data is received within 400 ms. |
| 8 | | 100007 | Driver connection lost |
| 9 | | 100008 | Encoder connection lost |
| 10 | | 100009 | DSP connection loss. For example, no DSP data is received within 40 s. |
| 11 | | 100010 | The communication link between the obstacle detection box and main control |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| | | | encountered error. |
| 12 | Behavior fault code | 400001 | Over 3° angle deviation when walking straight |
| 13 | | 400002 | The difference between the rotation integration angle and the encoder angle is out of limit. |
| 14 | | 400003 | The shift of the robot center position exceeds the limit. |
| 15 | | 400004 | The driver wheel slips when rotating. |
| 16 | | 400005 | The direction indicated by the control instruction is inconsistent with the actual angle rotation direction. For example, the fault might be caused by wrong instruction or excessive load. |
| 17 | | 400006 | During arc fitting based on points, the cumulative error of trajectory smoothness exceeds the limit. |
| 18 | | 400007 | In the scenario of entering the field grid, the error between the target posture and the stop posture exceeds the limit. |
| 19 | | 400008 | The rotation angle of the shelf exceeds 180°. |
| 20 | | 400009 | The relative error between the shelf QR code and the floor QR code exceeds the limit. |
| 21 | | 400010 | The shelf is lifted askew. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 22 | | 400011 | Gyro integration detection is carried out when the robot is in the stop mode. If the robot in the stop status is rotated by an external force, an error might be reported. |
| 23 | | 400012 | Image fusion angle change detection is carried out when the robot is in the stop mode. If the robot in the stop status is rotated by an external force, an error might be reported. |
| 24 | | 400013 | Encoder angle change detection is carried out when the robot is in the stop mode. If the robot in the stop status is rotated by an external force, an error might be reported. |
| 25 | | 400014 | The location deviation of x and y coordinates is 20 mm when the robot walks to the charging station. |
| 26 | | 400015 | The charging station location deviation of x and y integer coordinates is 20 mm. |
| 27 | | 400016 | Incorrect backward direction for charging |
| 28 | | 400017 | Height parameter error during lifting |
| 29 | | 400018 | Height parameter error during dropping |
| 30 | | 400019 | The distance between the current location of the robot and the start location of the path exceeds the limit. |
| 31 | | 400020 | When adjusting the shelf while rotating, the camera takes a picture of the shelf not in the right place. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 32 | | 400021 | The shelf is lifted askew. |
| 33 | | 400022 | null |
| 34 | | 400024 | Timeout in acquiring dsp data of ceil rect |
| 35 | | 400025 | Timeout in acquiring dsp data of ceil decode |
| 36 | | 400026 | Angle error in planning small paths |
| 37 | | 400027 | An error occurred when the robot was instructed to implement the task in manual mode. |
| 38 | Task fault code | 600001 | Switching of the robot in task status encountered error. |
| 39 | | 600002 | A wrong instruction was given at the end of the task. |
| 40 | | 600003 | Task step stage error |
| 41 | Driver fault code | 500001 | Driver instruction error |
| 42 | | 500002 | Driver motor phase error |
| 43 | | 500003 | Driver tracing error |
| 44 | | 500004 | Driver feedback error |
| 45 | | 500005 | Driver undervoltage |
| 46 | | 500006 | Driver overvoltage |
| 47 | | 500007 | Driver overcurrent |
| 48 | | 500008 | Driver short circuit |
| 49 | | 500009 | Driver motor overtemperature |
| 50 | | 500010 | Driver motor undertemperature |
| 51 | | 500011 | Driver motor overtemperature |
| 52 | | 500012 | Driver motor undertemperature |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 53 | | 500013 | driver overspeed alarm |
| 54 | | 500014 | Driver address error |
| 55 | Module fault code | 200001 | Driver instruction error |
| 56 | | 200002 | There is a big difference between the reference values of the two calibrations (before and after) performed by the gyroscope. |
| 57 | | 200003 | The driver wheel slips when rotating. |
| 58 | | 200004 | No update of obstacle detection data is received within 2 s. |
| 59 | | 200005 | No battery data update is received within 200 s. |
| 60 | | 200006 | The driver wheel is locked up. For example, if the driver wheels do not move for more than 2 seconds in the motion modes of straight line, arc line, rotation, etc., it is considered that they are locked. |
| 61 | | 200007 | The left wheel slips when the robot moves in a straight line. |
| 62 | | 200008 | The right wheel slips when the robot moves in a straight line. |
| 63 | | 200009 | An error occurred when saving images, such as calibration error of the floor QR code > 10° and side error > 4 cm. |
| 64 | | 200010 | Image feedback in non-QR code area, such |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| | | | as error reporting when a QR code is lost |
| 65 | | 200011 | The elevator mechanism fails to perform the lifting action. For example, an error is reported when the lifting motor does not lift the pallet within 20 seconds in the lifting status. |
| 66 | | 200012 | The pulses of the driver wheel encoder have not been updated. For example, the encoder malfunctions when the robot is in linear or arc motion. |
| 67 | | 200013 | The pulses of the driver wheel encoder overflow, that is, the count exceeds the upper limit. |
| 68 | | 200014 | Front physical anti-collision bar triggered |
| 69 | | 200015 | The QR code of the shelf is decoded incorrectly. For example, the black box has been identified, but the code value is wrong |
| 70 | | 200016 | Rear physical anti-collision bar triggered |
| 71 | | 200017 | Obstacle detection is triggered |
| 72 | | 200018 | The head tail checksum of the image data uploaded by the camera failed to be parsed. |
| 73 | | 200019 | The head and tail checksum of the image data uploaded by the camera failed to be parsed. |
| 74 | | 200020 | Charging without current |
| 75 | | 200021 | Charging sensor malfunctioned |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Classification | Reason code | Description |
|---|---|---|---|
| 76 | | 200022 | The original data of obstacle detection has not been updated within 2 seconds. |
| 77 | | 200023 | Driver wheel charging overcurrent |
| 78 | | 200024 | Driver wheel motor overcurrent |
| 79 | | 200025 | Lifting motor overcurrent |
| 80 | | 200026 | No DSP heartbeat. No feedback from the correct QR code position. |
| 81 | | 200027 | DSP data feedback error |
| 82 | | 200028 | Shelf QR code decoding failed. For example, even the black box is not parsed. |
| 83 | | 200029 | Shelf QR code decoding timeout |
| 84 | | 200030 | Driver over-temperature |

## 8.4 System Exception Codes

| No. | Fault code | Fault description |
|---|---|---|
| 1 | lang.mwms.monitorRobotMsg.config | Workflow configuration encountered error. Please check the workflow configuration. |
| 2 | lang.mwms.monitorRobotMsg.norobot | No available robots |
| 3 | lang.mwms.monitorRobotMsg.noshelf | No available shelves |
| 4 | lang.mwms.monitorRobotMsg.notfree | The shelf is not idle. |
| 5 | lang.mwms.monitorRobotMsg.other | Other exception |
| 6 | lang.mwms.monitorRobotMsg.scanabnormal | Code scanning exception |
| 7 | lang.mwms.monitorRobotMsg.sendtaskfail | Sending the task to RMS encountered error. Please check |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| | | the operating status of RMS. |
| 8 | lang.mwms.monitorRobotMsg.targetnotfree | The destination location (area) is not idle. |
| 9 | lang.mwms.monitorRobotMsg1 | Path planning failed. |
| 10 | lang.mwms.monitorRobotMsg10 | Task exception |
| 11 | lang.mwms.monitorRobotMsg10000 | Big temperature changes of gyroscope |
| 12 | lang.mwms.monitorRobotMsg10001 | There is a big difference between the reference values of the two calibrations (before and after) performed by the gyroscope. |
| 13 | lang.mwms.monitorRobotMsg10002 | The driver wheel slips when rotating. |
| 14 | lang.mwms.monitorRobotMsg10003 | No update of obstacle detection data is received within 2 s. |
| 15 | lang.mwms.monitorRobotMsg10004 | No battery data update is received within 200 s. |
| 16 | lang.mwms.monitorRobotMsg10005 | The driver wheel is locked up. For example, if the driver wheels do not move for more than 2 seconds in the motion modes of straight line, arc line, rotation, etc., it is considered that they are locked. |
| 17 | lang.mwms.monitorRobotMsg10006 | The left wheel slips when the |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| | | robot moves in a straight line. |
| 18 | lang.mwms.monitorRobotMsg10007 | The right wheel slips when the robot moves in a straight line. |
| 19 | lang.mwms.monitorRobotMsg10008 | An error occurred when saving images, such as calibration error of the floor QR code >10°, side error > 4cm (not applied). |
| 20 | lang.mwms.monitorRobotMsg10009 | Image feedback in non-QR code area, such as error reporting when a QR code is lost |
| 21 | lang.mwms.monitorRobotMsg10010 | The elevator mechanism fails to perform the lifting action. For example, an error is reported when the lifting motor does not lift the pallet within 20 seconds in the lifting status. |
| 22 | lang.mwms.monitorRobotMsg10011 | The pulses of the driver wheel encoder have not been updated. For example, the encoder malfunctions when the robot is in linear or arc motion. |
| 23 | lang.mwms.monitorRobotMsg10012 | The pulses of the driver wheel encoder overflow, that is, the count exceeds the upper limit. |
| 24 | lang.mwms.monitorRobotMsg10013 | Front physical anti-collision bar triggered |
| 25 | lang.mwms.monitorRobotMsg10014 | Shelf QR code decoding error |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| 26 | lang.mwms.monitorRobotMsg10015 | Rear physical anti-collision bar triggered |
| 27 | lang.mwms.monitorRobotMsg10016 | Obstacle detection triggered |
| 28 | lang.mwms.monitorRobotMsg10017 | The lifting motor cannot descend For example, it fails to drop the shelf within 40 s. |
| 29 | lang.mwms.monitorRobotMsg10018 | The head and tail checksum of the image data uploaded by the camera failed to be parsed. |
| 30 | lang.mwms.monitorRobotMsg10019 | Charging without current |
| 31 | lang.mwms.monitorRobotMsg10020 | Charging sensor malfunctioned |
| 32 | lang.mwms.monitorRobotMsg10021 | The original data of obstacle detection has not been updated within 2 seconds. |
| 33 | lang.mwms.monitorRobotMsg10022 | Driver wheel charging overcurrent, non-faulty |
| 34 | lang.mwms.monitorRobotMsg10023 | Driver wheel motor overcurrent |
| 35 | lang.mwms.monitorRobotMsg10024 | Lifting motor overcurrent |
| 36 | lang.mwms.monitorRobotMsg10025 | No DSP heartbeat. No feedback from the correct QR code position. |
| 37 | lang.mwms.monitorRobotMsg10026 | DSP data feedback error |
| 38 | lang.mwms.monitorRobotMsg10027 | Shelf QR code decoding failed. For example, even the black box is not parsed. |
| 39 | lang.mwms.monitorRobotMsg10028 | Shelf QR code decoding timeout |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| **40** | lang.mwms.monitorRobotMsg10029 | Driver over-temperature |
| **41** | lang.mwms.monitorRobotMsg10030 | The obstacle detection box/host computer does not receive data from the obstacle detector. |
| **42** | lang.mwms.monitorRobotMsg10031 | The communication link between the obstacle detection box and main control encountered error. |
| **43** | lang.mwms.monitorRobotMsg10032 | The obstacle detection box/host computer received incorrect data from the obstacle detector. |
| **44** | lang.mwms.monitorRobotMsg10033 | The communication link between the power module and main control encountered error. |
| **45** | lang.mwms.monitorRobotMsg10034 | The power module received incorrect battery data. |
| **46** | lang.mwms.monitorRobotMsg10035 | CAN1 communication error. For example, no data is received within 400 ms. |
| **47** | lang.mwms.monitorRobotMsg10036 | CAN2 communication error. For example, no data is received within 400 ms. |
| **48** | lang.mwms.monitorRobotMsg10037 | Driver connection lost |
| **49** | lang.mwms.monitorRobotMsg10038 | Encoder connection lost |
| **50** | lang.mwms.monitorRobotMsg10039 | DSP connection loss. For example, no DSP data is received within 40 s. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| 51 | lang.mwms.monitorRobotMsg10040 | The side deviation when the robot walks in a straight line exceeds 20 mm. |
| 52 | lang.mwms.monitorRobotMsg10041 | Over 3° angle deviation when walking straight |
| 53 | lang.mwms.monitorRobotMsg10042 | The difference between the rotation integration angle and the encoder angle is out of limit. |
| 54 | lang.mwms.monitorRobotMsg10043 | The shift of the center position exceeds the limit. |
| 55 | lang.mwms.monitorRobotMsg10044 | The driver wheel slips when rotating. |
| 56 | lang.mwms.monitorRobotMsg10045 | The direction indicated by the control instruction is inconsistent with the actual angle rotation direction. For example, the fault might be caused by wrong instruction or excessive load. |
| 57 | lang.mwms.monitorRobotMsg10046 | During arc fitting based on points, the cumulative error of trajectory smoothness exceeds the limit. |
| 58 | lang.mwms.monitorRobotMsg10047 | In the scenario of entering the field grid, the error between the target posture and the stop posture exceeds the limit. |
| 59 | lang.mwms.monitorRobotMsg10048 | The rotation angle of the shelf |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| | | exceeds 180°. |
| **60** | lang.mwms.monitorRobotMsg10049 | The relative error between the shelf QR code and the floor QR code exceeds the limit. |
| **61** | lang.mwms.monitorRobotMsg10050 | The shelf is lifted askew. |
| **62** | lang.mwms.monitorRobotMsg10051 | Gyro integration detection is carried out in the stop mode. If the robot in the stop status is rotated by an external force, an error might be reported. |
| **63** | lang.mwms.monitorRobotMsg10052 | Image fusion angle change detection is carried out in the stop mode. |
| **64** | lang.mwms.monitorRobotMsg10053 | Encoder angle change detection is carried out in the stop mode. If the robot in the stop status is rotated by an external force, an error might be reported. |
| **65** | lang.mwms.monitorRobotMsg10054 | The location deviation of x and y coordinates is 20 mm when the robot walks to the charging station. |
| **66** | lang.mwms.monitorRobotMsg10055 | The charging station location deviation of x and y integer coordinates is 20 mm. |
| **67** | lang.mwms.monitorRobotMsg10056 | Incorrect backward direction for charging |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| 68 | lang.mwms.monitorRobotMsg10057 | Height parameter error during lifting |
| 69 | lang.mwms.monitorRobotMsg10058 | Height parameter error during dropping |
| 70 | lang.mwms.monitorRobotMsg10059 | The distance between the current location and the start location of the path exceeds the limit. |
| 71 | lang.mwms.monitorRobotMsg10060 | When adjusting the shelf while rotating, the camera takes a picture of the shelf not in the right place. |
| 72 | lang.mwms.monitorRobotMsg10061 | If a shelf QR code position is very poor, do not put the shelf down to prevent hitting the shelf next to it |
| 73 | lang.mwms.monitorRobotMsg10062 | More than 2 floor QR codes lost |
| 74 | lang.mwms.monitorRobotMsg10063 | Timeout in acquiring dsp data of ceilrect |
| 75 | lang.mwms.monitorRobotMsg10064 | Timeout in acquiring dsp data of ceildecode |
| 76 | lang.mwms.monitorRobotMsg10065 | Angle error in planning small paths |
| 77 | lang.mwms.monitorRobotMsg10066 | Switching of the robot in task status encountered error. |
| 78 | lang.mwms.monitorRobotMsg10067 | A wrong instruction was given at |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| | | the end of the task. |
| 79 | lang.mwms.monitorRobotMsg10068 | Task step stage error |
| 80 | lang.mwms.monitorRobotMsg10069 | Driver instruction error |
| 81 | lang.mwms.monitorRobotMsg10070 | Driver motor phase error |
| 82 | lang.mwms.monitorRobotMsg10071 | Driver tracing error |
| 83 | lang.mwms.monitorRobotMsg10072 | Driver feedback error |
| 84 | lang.mwms.monitorRobotMsg10073 | Driver undervoltage |
| 85 | lang.mwms.monitorRobotMsg10074 | Driver overvoltage |
| 86 | lang.mwms.monitorRobotMsg10075 | Driver overcurrent |
| 87 | lang.mwms.monitorRobotMsg10076 | Driver short circuit |
| 88 | lang.mwms.monitorRobotMsg10077 | Driver motor overtemperature |
| 89 | lang.mwms.monitorRobotMsg10078 | Driver motor undertemperature |
| 90 | lang.mwms.monitorRobotMsg10079 | Driver motor overtemperature |
| 91 | lang.mwms.monitorRobotMsg10080 | Driver motor undertemperature |
| 92 | lang.mwms.monitorRobotMsg10081 | driver overspeed alarm |
| 93 | lang.mwms.monitorRobotMsg10082 | Driver address error |
| 94 | lang.mwms.monitorRobotMsg12000 | Robot connection failed. |
| 95 | lang.mwms.monitorRobotMsg12001 | Subtask sending timeout |
| 96 | lang.mwms.monitorRobotMsg12002 | Path planning failed. |
| 97 | lang.mwms.monitorRobotMsg12003 | The robot is not on the map. |
| 98 | lang.mwms.monitorRobotMsg12004 | The start point or end point of the task is an obstacle. |
| 99 | lang.mwms.monitorRobotMsg12005 | There are robots or shelves in the path which are obstacles. |
| 100 | lang.mwms.monitorRobotMsg12006 | The path resources are likely |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| | | being occupied. |
| **101** | lang.mwms.monitorRobotMsg12007 | No path can be planned. |
| **102** | lang.mwms.monitorRobotMsg12009 | Timeout of shelf fetching by robot |
| **103** | lang.mwms.monitorRobotMsg12010 | Fetch the shelf before reaching the shelf location. |
| **104** | lang.mwms.monitorRobotMsg12011 | Failed to arrive at the wait point after a long time. |
| **105** | lang.mwms.monitorRobotMsg12012 | The battery level of the robot does not increase after a long time. |
| **106** | lang.mwms.monitorRobotMsg12013 | The robot reaches the low electric ratio. |
| **107** | lang.mwms.monitorRobotMsg12014 | Failed to implement the charging task because the charging station does not match. |
| **108** | lang.mwms.monitorRobotMsg12015 | Failed to implement the charging task because of no idle charging station. |
| **109** | lang.mwms.monitorRobotMsg12016 | Failed to implement the charging task because of unavailable charging station. |
| **110** | lang.mwms.monitorRobotMsg12017 | The charging station cell is occupied by other robots. |
| **111** | lang.mwms.monitorRobotMsg12018 | Failed to charge the robot across the area. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| 112 | lang.mwms.monitorRobotMsg12019 | Failed to charge the robot across the floor. |
| 113 | lang.mwms.monitorRobotMsg12020 | Failed to implement the charging task because the current task is blocked. |
| 114 | lang.mwms.monitorRobotMsg12021 | Failed to implement the charging task because the current task is not finished. |
| 115 | lang.mwms.monitorRobotMsg12022 | The charging time is too long. |
| 116 | lang.mwms.monitorRobotMsg12023 | The battery temperature of robot is too high. |
| 117 | lang.mwms.monitorRobotMsg12024 | Failed to find the rest location. |
| 118 | lang.mwms.monitorRobotMsg13000 | Charging station disconnection (temporary network interruption) |
| 119 | lang.mwms.monitorRobotMsg13001 | Charging station disconnection (loss of communication for a long time) |
| 120 | lang.mwms.monitorRobotMsg13002 | Timeout for sending the charging station start instruction |
| 121 | lang.mwms.monitorRobotMsg13003 | Timeout for sending the charging station ending instruction |
| 122 | lang.mwms.monitorRobotMsg14000 | The shelf location is to be confirmed. |
| 123 | lang.mwms.monitorRobotMsg14004 | Driver undervoltage |
| 124 | lang.mwms.monitorRobotMsg16000 | Robot abnormal |
| 125 | lang.mwms.monitorRobotMsg2 | The robot is not on the map. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| 126 | lang.mwms.monitorRobotMsg21000 | The side deviation for end QR code positioning is more than 20 mm; the angle deviation is more than 2°. |
| 127 | lang.mwms.monitorRobotMsg21001 | Over 3° angle deviation when walking straight |
| 128 | lang.mwms.monitorRobotMsg21002 | The difference between the rotation integration angle and the encoder angle is out of limit. |
| 129 | lang.mwms.monitorRobotMsg21003 | The shift of the robot center position exceeds the limit. |
| 130 | lang.mwms.monitorRobotMsg21004 | The driver wheel slips when rotating. |
| 131 | lang.mwms.monitorRobotMsg21005 | The direction indicated by the control instruction is inconsistent with the actual angle rotation direction. For example, the fault might be caused by wrong instruction or excessive load. |
| 132 | lang.mwms.monitorRobotMsg21006 | During arc fitting based on points, the cumulative error of trajectory smoothness exceeds the limit. |
| 133 | lang.mwms.monitorRobotMsg21007 | In the scenario of entering the field grid, the error between the target posture and the stop posture exceeds the limit. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| 134 | lang.mwms.monitorRobotMsg21008 | The rotation angle of the shelf exceeds 180°. |
| 135 | lang.mwms.monitorRobotMsg21009 | The relative error between the shelf QR code and the floor QR code exceeds the limit. |
| 136 | lang.mwms.monitorRobotMsg21010 | The shelf is lifted askew. |
| 137 | lang.mwms.monitorRobotMsg21011 | Gyro integration detection is carried out when the robot is in the stop mode. If the robot in the stop status is rotated by an external force, an error might be reported. |
| 138 | lang.mwms.monitorRobotMsg21012 | Image fusion angle change detection is carried out when the robot is in the stop mode. If the robot in the stop status is rotated by an external force, an error might be reported. |
| 139 | lang.mwms.monitorRobotMsg21013 | Encoder angle change detection is carried out when the robot is in the stop mode. If the robot in the stop status is rotated by an external force, an error might be reported. |
| 140 | lang.mwms.monitorRobotMsg21014 | The location deviation of x and y coordinates is 20 mm when the robot walks to the charging |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| | | station. |
| 141 | lang.mwms.monitorRobotMsg21015 | The charging station location deviation of x and y integer coordinates is 20 mm. |
| 142 | lang.mwms.monitorRobotMsg21016 | Incorrect backward direction for charging |
| 143 | lang.mwms.monitorRobotMsg21017 | Height parameter error during lifting |
| 144 | lang.mwms.monitorRobotMsg21018 | Height parameter error during dropping |
| 145 | lang.mwms.monitorRobotMsg21019 | The distance between the current location of the robot and the start location of the path exceeds the limit. |
| 146 | lang.mwms.monitorRobotMsg21020 | When adjusting the shelf while rotating, the camera takes a picture of the shelf not in the right place. |
| 147 | lang.mwms.monitorRobotMsg21021 | If a shelf QR code position is very poor, do not put the shelf down to prevent hitting the shelf next to it |
| 148 | lang.mwms.monitorRobotMsg21022 | More than 2 floor QR codes lost |
| 149 | lang.mwms.monitorRobotMsg21023 | Timeout in acquiring dsp data of ceil rect |
| 150 | lang.mwms.monitorRobotMsg21024 | Timeout in acquiring dsp data of |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| | | ceil decode |
| 151 | lang.mwms.monitorRobotMsg21025 | Angle error in planning small paths |
| 152 | lang.mwms.monitorRobotMsg21026 | An error occurred when the robot was instructed to implement the task in manual mode. |
| 153 | lang.mwms.monitorRobotMsg21027 | Charging failed. |
| 154 | lang.mwms.monitorRobotMsg21028 | The shelf QR code is not within the field of view during robot walking |
| 155 | lang.mwms.monitorRobotMsg21029 | The first floor QR code cannot be scanned after switching to QR code navigation. |
| 156 | lang.mwms.monitorRobotMsg21030 | The wheel slips during movement. |
| 157 | lang.mwms.monitorRobotMsg21031 | Exception occurred when recognizing the shelf model. |
| 158 | lang.mwms.monitorRobotMsg21032 | Emergency stop button is triggered. |
| 159 | lang.mwms.monitorRobotMsg21033 | The manual mode is enabled. |
| 160 | lang.mwms.monitorRobotMsg21034 | The obstacle detection box/host computer does not receive data from the obstacle detector. |
| 161 | lang.mwms.monitorRobotMsg21035 | The communication link between the obstacle detection box and |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| | | main control encountered error. |
| **162** | lang.mwms.monitorRobotMsg21036 | The obstacle detection box/host computer received incorrect data from the obstacle detector. |
| **163** | lang.mwms.monitorRobotMsg21037 | The communication link between the power module and main control encountered error. |
| **164** | lang.mwms.monitorRobotMsg21038 | The power module received incorrect battery data. |
| **165** | lang.mwms.monitorRobotMsg21039 | CAN1 communication error. For example, no data is received within 400 ms. |
| **166** | lang.mwms.monitorRobotMsg21040 | CAN2 communication error. For example, no data is received within 400 ms. |
| **167** | lang.mwms.monitorRobotMsg21041 | Driver connection lost |
| **168** | lang.mwms.monitorRobotMsg21042 | Encoder connection lost |
| **169** | lang.mwms.monitorRobotMsg21043 | DSP connection loss. For example, no DSP data is received within 40 s. |
| **170** | lang.mwms.monitorRobotMsg21044 | Big temperature changes of gyroscope |
| **171** | lang.mwms.monitorRobotMsg21045 | There is a big difference between the reference values of the two calibrations (before and after) performed by the |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| | | gyroscope. |
| 172 | lang.mwms.monitorRobotMsg21046 | The driver wheel slips when rotating. |
| 173 | lang.mwms.monitorRobotMsg21047 | No update of obstacle detection data is received within 2 s. |
| 174 | lang.mwms.monitorRobotMsg21048 | No battery data update is received within 200 s. |
| 175 | lang.mwms.monitorRobotMsg21049 | The driver wheel is locked up. For example, if the driver wheels do not move for more than 2 seconds in the motion modes of straight line, arc line, rotation, etc., it is considered that they are locked. |
| 176 | lang.mwms.monitorRobotMsg21050 | The left wheel slips when the robot moves in a straight line. |
| 177 | lang.mwms.monitorRobotMsg21051 | The right wheel slips when the robot moves in a straight line. |
| 178 | lang.mwms.monitorRobotMsg21052 | An error occurred when saving images, such as calibration error of the floor QR code > 10° and side error > 4 cm. |
| 179 | lang.mwms.monitorRobotMsg21053 | Image feedback in non-QR code area, such as error reporting when a QR code is lost |
| 180 | lang.mwms.monitorRobotMsg21054 | The elevator mechanism fails to perform the lifting action. For |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| | | example, an error is reported when the lifting motor does not lift the pallet within 20 seconds in the lifting status. |
| 181 | lang.mwms.monitorRobotMsg21055 | The pulses of the driver wheel encoder have not been updated. For example, the encoder malfunctions when the robot is in linear or arc motion. |
| 182 | lang.mwms.monitorRobotMsg21056 | The pulses of the driver wheel encoder overflow, that is, the count exceeds the upper limit. |
| 183 | lang.mwms.monitorRobotMsg21057 | Front physical anti-collision bar triggered |
| 184 | lang.mwms.monitorRobotMsg21058 | The QR code of the shelf is decoded incorrectly. For example, the black box has been identified, but the code value is wrong |
| 185 | lang.mwms.monitorRobotMsg21059 | Rear physical anti-collision bar triggered |
| 186 | lang.mwms.monitorRobotMsg21060 | Robot obstacle detection |
| 187 | lang.mwms.monitorRobotMsg21061 | The head tail checksum of the image data uploaded by the camera failed to be parsed. |
| 188 | lang.mwms.monitorRobotMsg21062 | The head and tail checksum of the image data uploaded by the |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| | | camera failed to be parsed. |
| 189 | lang.mwms.monitorRobotMsg21063 | Charging without current |
| 190 | lang.mwms.monitorRobotMsg21064 | Charging sensor malfunctioned |
| 191 | lang.mwms.monitorRobotMsg21065 | The original data of obstacle detection has not been updated within 2 seconds. |
| 192 | lang.mwms.monitorRobotMsg21066 | Driver wheel charging overcurrent |
| 193 | lang.mwms.monitorRobotMsg21067 | Driver wheel motor overcurrent |
| 194 | lang.mwms.monitorRobotMsg21068 | Lifting motor overcurrent |
| 195 | lang.mwms.monitorRobotMsg21069 | No DSP heartbeat. No feedback from the correct QR code position. |
| 196 | lang.mwms.monitorRobotMsg21070 | DSP data feedback error |
| 197 | lang.mwms.monitorRobotMsg21071 | Shelf QR code decoding failed. For example, even the black box is not parsed. |
| 198 | lang.mwms.monitorRobotMsg21072 | Shelf QR code decoding timeout |
| 199 | lang.mwms.monitorRobotMsg21073 | Shelf QR code decoding timeout |
| 200 | lang.mwms.monitorRobotMsg21074 | Loss of lidar data or device fault |
| 201 | lang.mwms.monitorRobotMsg21075 | Loss of battery data |
| 202 | lang.mwms.monitorRobotMsg21076 | Battery over-temperature protection |
| 203 | lang.mwms.monitorRobotMsg21077 | Recoverable motor module fault |
| 204 | lang.mwms.monitorRobotMsg21078 | Irrecoverable motor module fault (replacement) |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| 205 | lang.mwms.monitorRobotMsg21079 | Low battery level for the absolute encoder |
| 206 | lang.mwms.monitorRobotMsg21080 | Loss of weighing sensor data |
| 207 | lang.mwms.monitorRobotMsg21081 | Communication interruption between main control and industrial PC |
| 208 | lang.mwms.monitorRobotMsg21082 | Switching of the robot in task status encountered error. |
| 209 | lang.mwms.monitorRobotMsg21083 | A wrong instruction was given at the end of the task. |
| 210 | lang.mwms.monitorRobotMsg21084 | Task step stage error |
| 211 | lang.mwms.monitorRobotMsg21085 | Driver instruction error |
| 212 | lang.mwms.monitorRobotMsg21086 | Driver motor phase error |
| 213 | lang.mwms.monitorRobotMsg21087 | Driver tracing error |
| 214 | lang.mwms.monitorRobotMsg21088 | Driver feedback error |
| 215 | lang.mwms.monitorRobotMsg21089 | Driver undervoltage |
| 216 | lang.mwms.monitorRobotMsg21090 | Driver overvoltage |
| 217 | lang.mwms.monitorRobotMsg21091 | Driver overcurrent |
| 218 | lang.mwms.monitorRobotMsg21092 | Driver short circuit |
| 219 | lang.mwms.monitorRobotMsg21093 | Driver motor overtemperature |
| 220 | lang.mwms.monitorRobotMsg21094 | Driver motor undertemperature |
| 221 | lang.mwms.monitorRobotMsg21095 | Driver motor overtemperature |
| 222 | lang.mwms.monitorRobotMsg21096 | Driver motor undertemperature |
| 223 | lang.mwms.monitorRobotMsg21097 | driver overspeed alarm |
| 224 | lang.mwms.monitorRobotMsg21098 | Driver address error |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| 225 | lang.mwms.monitorRobotMsg21099 | Component delivery exception |
| 226 | lang.mwms.monitorRobotMsg21100 | Component receipt exception |
| 227 | lang.mwms.monitorRobotMsg21101 | Component communication interruption |
| 228 | lang.mwms.monitorRobotMsg21102 | Component fault |
| 229 | lang.mwms.monitorRobotMsg21103 | Loss of depth camera data |
| 230 | lang.mwms.monitorRobotMsg21104 | Loss of fisheye camera data |
| 231 | lang.mwms.monitorRobotMsg21105 | Network interruption |
| 232 | lang.mwms.monitorRobotMsg21106 | Loss of driver data or device fault |
| 233 | lang.mwms.monitorRobotMsg21107 | STO triggered |
| 234 | lang.mwms.monitorRobotMsg21108 | Press the brake release. |
| 235 | lang.mwms.monitorRobotMsg21109 | Battery runs out. |
| 236 | lang.mwms.monitorRobotMsg21110 | Overweight or excessive unbalanced load |
| 237 | lang.mwms.monitorRobotMsg21111 | Positioning loss |
| 238 | lang.mwms.monitorRobotMsg22000 | RMS communication interruption |
| 239 | lang.mwms.monitorRobotMsg22001 | CAN communication interruption (charging module) |
| 240 | lang.mwms.monitorRobotMsg22002 | Screen communication interruption |
| 241 | lang.mwms.monitorRobotMsg22003 | RMS data exception |
| 242 | lang.mwms.monitorRobotMsg22004 | RMS command exception |
| 243 | lang.mwms.monitorRobotMsg22005 | Press emergency stop of the forklift charging station. |

This document shall not be disseminated in any form without the written permission of Geek+.

| No. | Fault code | Fault description |
|---|---|---|
| 244 | lang.mwms.monitorRobotMsg22006 | No sensor is detected in the forklift charging station. |
| 245 | lang.mwms.monitorRobotMsg22007 | Charging module over-temperature |
| 246 | lang.mwms.monitorRobotMsg22008 | The charging current is 0 in auto mode. |
| 247 | lang.mwms.monitorRobotMsg22009 | Charging module alarm status |
| 248 | lang.mwms.monitorRobotMsg22010 | Charging module error status |
| 249 | lang.mwms.monitorRobotMsg22011 | The charging station reports its unavailability. |
| 250 | lang.mwms.monitorRobotMsg3 | The start point and end point of the planned path are obstacles. |
| 251 | lang.mwms.monitorRobotMsg4 | There are robots or shelves in the path which are obstacles. |
| 252 | lang.mwms.monitorRobotMsg5 | The current path resources are being occupied by other robots. |
| 253 | lang.mwms.monitorRobotMsg6 | No path can be planned. |
| 254 | lang.mwms.monitorRobotMsg7 | Robot connection failed. |
| 255 | lang.mwms.monitorRobotMsg8 | Subtask sending timeout |
| 256 | lang.mwms.monitorRobotMsg81000 | The batter level of robots across the field is relatively low. |
| 257 | lang.mwms.monitorRobotMsg9 | Instruction sending timeout |
| 258 | lang.mwms.monitorRobotMsg91000 | Emergency stop button triggered |
| 259 | lang.mwms.monitorRobotMsg91001 | Emergency stop button disconnected |

This document shall not be disseminated in any form without the written permission of Geek+.